FullFlow: Upgrading Text-to-Image Flow Matching Models for Bidirectional Vision–Language Generation

Anonymous Author(s)
Affiliation
Address
email

Abstract

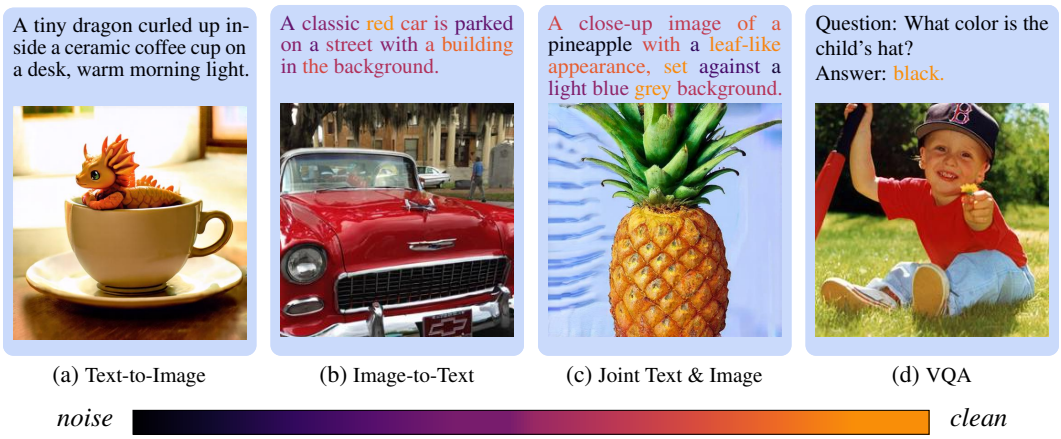
Modern text-to-image diffusion models encode rich visual priors, but expose them only through one-way text-conditioned generation. Existing unified vision–language models derived from them recover bidirectional capability through large-scale joint pretraining or substantial retraining of the text pathway, discarding the strong image prior the text-to-image backbone already encodes. We introduce *FullFlow*, a parameter-efficient recipe that upgrades a pretrained rectified-flow text-to-image model into a bidirectional vision–language generator by training only LoRA adapters and lightweight text heads. FullFlow keeps images in their native continuous flow and adds a discrete insertion process for text. Separate image and text timesteps turn inference into trajectory selection in a two-dimensional generative space, enabling text→image, image→text, joint sampling, and partial-text prediction with a single backbone. On Stable Diffusion 3 (SD3) under an identical trainable-parameter count and matched LoRA rank, FullFlow improves text→image FID from 62.7 to 31.6 and image→text CIDEr from 2.0 to 99.4 over a LoRA equivalent following the previous SOTA formulation (Dual Diffusion) at matched wall-clock training time, while reducing peak VRAM from ~ 84 GB to ~ 38 GB and raising throughput by $\sim 8\times$ on two RTX A5000 GPUs in under 24 hours, training only $\sim 5\%$ of the backbone parameters. The same recipe transfers to FLUX.1-dev and supports downstream VQA through partial-text generation. These results show that strong bidirectional vision–language capability can be unlocked from pretrained text-to-image flow models without full multimodal pretraining.

1 Introduction

Pretrained text-to-image flow models are strong image generators, yet they work in only one direction: text in, image out. Diffusion established the paradigm for high-fidelity synthesis [20, 45], and rectified-flow transformers further improved scalability, prompt fidelity, and compositional generation [43, 12, 32, 35, 25]. Their internal representations are deeply structured: attention localizes entities [8, 24], token interventions affect attribute binding [18, 6, 40], and benchmarks confirm increasingly rich object-relation modeling [22]. A backbone that has learned how words, objects, and spatial layouts co-vary should, in principle, also support the inverse direction: describing an image, answering questions about it, or sampling coherent image–text pairs.

Exploiting this latent capability is not straightforward. Most vision–language models (VLMs) are built in the opposite direction: a pretrained language model is given a visual encoder, making vision a conditioning signal for autoregressive text generation [2, 28, 34, 62, 11]. Unified generators that combine autoregressive text with diffusion-based images [61, 38] are similarly language-first. These designs discard the strong image prior the text-to-image backbone already encodes, and rebuilding it requires expensive joint pretraining. This raises a natural question: can a pretrained text-to-image flow model be made bidirectional without retraining from scratch?

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.



(a) Text-to-Image (b) Image-to-Text (c) Joint Text & Image (d) VQA

Figure 1: Overview of the multimodal generation capabilities after finetuning Stable Diffusion 3. Beyond standard text-to-image synthesis, the model enables image-to-text generation, joint sampling over text and image and Visual Question Answering. The color bar visualizes text diffusion time.

We study this question in the **lightweight-upgrade regime**: starting from an SD3- or FLUX-style generator, preserving its pretrained image process, and adding text generation with minimal new parameters. This regime is both scientifically motivated, as a test of whether the visual-semantic structure already encoded by text-to-image models is sufficient for image-to-text, joint, and partial-text generation, and practically motivated: full multimodal pretraining is prohibitively expensive, whereas a low-cost recipe places follow-up experimentation on bidirectional rectified-flow models within reach of the broader research community. Prior unified diffusion models show that joint generation is possible, but operate in different regimes: end-to-end multimodal pretraining [5, 61, 41], shared discrete or flow-based representations [52], or substantial backbone retraining [31].

We propose *FullFlow*, a parameter-efficient recipe for upgrading pretrained rectified-flow text-to-image models into bidirectional vision–language generators. The key insight is to treat the two modalities asymmetrically but compatibly: images remain in the native continuous rectified-flow latent space, while text is handled by a lightweight discrete insertion process built on Edit Flows [16, 13, 7]. A single shared backbone processes both modalities under decoupled image and text timesteps (t, τ), turning inference into trajectory selection in a two-dimensional generative space: fixing one modality recovers the conditional tasks; evolving both yields unconditional image–text sampling; constraining a target span enables partial-text prediction for tasks such as VQA. For SD3, only LoRA adapters and added text heads are trained [21] — $\sim 5\%$ of parameters — and the full upgrade fits in 24 hours on two commodity RTX A5000 GPUs.

Empirically, FullFlow turns strong text-to-image generators into bidirectional vision–language models without sacrificing the original image prior. On SD3 against a matched Dual Diffusion-LoRA [31] baseline at identical trainable-parameter count and LoRA rank, FullFlow improves text-to-image generation FID from 62.7 to 29.3 and image captioning CIDEr from 2.0 to 13.4 at matched gradient steps, with captioning CIDEr further reaching 99.4 at matched wall-clock time, at $\sim 8\times$ throughput and less than half the peak VRAM. The same recipe transfers to FLUX.1-dev, enables joint image–text sampling, and supports downstream VQA through the same insertion-based text interface.

FullFlow makes three contributions:

- **C1.** Converting a pretrained text-to-image rectified-flow model into a bidirectional vision–language generator by training only LoRA adapters and lightweight text heads, while preserving the pretrained image prior.
- **C2.** Keeping images in the native continuous flow, adding a discrete insertion process for text, and decoupling image and text timesteps so that conditional, joint, and partial-text generation become different trajectories in a single (t, τ) space.
- **C3.** On SD3 and FLUX.1-dev, outperforming a matched Dual Diffusion-LoRA baseline on both text→image retention and image→text quality and supporting downstream VQA, while training on two commodity GPUs.

2 Background

We frame both image and text generation in a single *path-based* view: each modality moves from a simple base distribution p_0 to the data distribution p_1 along a path indexed by time, and is trained to predict local transport along that path.

We use $t \in [0,1]$ for image time and $\tau \in [0,1]$ for text time, with 0 the base state and 1 clean data. Images (or latents) are denoted by $\boldsymbol{x} \in \mathcal{X} \subset \mathbb{R}^d$. Text is a variable-length token sequence $\boldsymbol{y} \in \mathcal{Y}$ with

$$\mathcal{Y} \coloneqq \bigcup_{L \geq 0} V^L \times \{\langle \texttt{EOS} \rangle\}, \tag{2.1}$$

where V is a fixed vocabulary and $\langle \texttt{EOS} \rangle$ is the end-of-sequence token. We write $\varepsilon = (\langle \texttt{EOS} \rangle)$ for the empty sequence; the discrete base distribution places all mass on ε.

2.1 Continuous Flow Matching for Images

In continuous space, a model defines a time-dependent vector field $v_\theta(t, \boldsymbol{x})$ and generates samples by integrating

$$\frac{d\boldsymbol{x}_t}{dt} = v_\theta(t, \boldsymbol{x}_t), \qquad t \in [0,1]. \tag{2.2}$$

Flow Matching [32] trains this vector field by specifying a path between a base sample $\boldsymbol{x}_0$ and a target sample $\boldsymbol{x}_1$, and then regressing onto the instantaneous velocity along that path. Concretely, given an interpolation $\boldsymbol{x}_t = \psi_t(\boldsymbol{x}_0, \boldsymbol{x}_1)$, the target velocity is

$$u(t, \boldsymbol{x}_t \mid \boldsymbol{x}_0, \boldsymbol{x}_1) = \frac{\partial}{\partial t} \psi_t(\boldsymbol{x}_0, \boldsymbol{x}_1), \tag{2.3}$$

and the model is trained with

$$\mathcal{L}_{\text{img}}(\theta) = \mathbb{E}_{t \sim \text{Unif}([0,1])} \mathbb{E}_{\substack{\boldsymbol{x}_0 \sim p_0 \\ \boldsymbol{x}_1 \sim p_1}} \Big[\| v_\theta(t, \boldsymbol{x}_t) - u(t, \boldsymbol{x}_t \mid \boldsymbol{x}_0, \boldsymbol{x}_1) \|_2^2 \Big], \tag{2.4}$$

where $\boldsymbol{x}_t = \psi_t(\boldsymbol{x}_0, \boldsymbol{x}_1)$. In this work, we use rectified flow [35], which uses the linear interpolation

$$\boldsymbol{x}_t = (1-t)\boldsymbol{x}_0 + t\boldsymbol{x}_1 \implies u(t, \boldsymbol{x}_t \mid \boldsymbol{x}_0, \boldsymbol{x}_1) = \boldsymbol{x}_1 - \boldsymbol{x}_0. \tag{2.5}$$

2.2 Discrete Flow Matching for Text

For text, the state space $\mathcal{Y}$ is discrete and variable-length, so generation cannot be described by a Euclidean velocity field. Instead, local transport is specified by transition rates between sequences. We model these transitions by a continuous-time Markov chain (CTMC) with time-dependent generator Q_τ, where $Q_\tau(\boldsymbol{y}, \boldsymbol{y}')$ is the instantaneous rate of jumping from sequence $\boldsymbol{y}$ to sequence $\boldsymbol{y}'$. Using the row-vector convention for distributions, the marginal law p_τ evolves according to

$$\frac{d}{d\tau} p_\tau = p_\tau Q_\tau. \tag{2.6}$$

Such jump processes provide a discrete analogue of flow-based generation [7, 13]: the generator Q_τ plays the role of a discrete velocity field by specifying the local direction of probability transport.

Following Edit Flows [16], we use a deletion-to-base forward process and learn the reverse insertion process. Given a clean sequence $\boldsymbol{y} \sim p_1$, we construct a corrupted subsequence $\tilde{\boldsymbol{y}}_\tau$ by independently retaining each non-$\langle \texttt{EOS} \rangle$ token with probability τ, while preserving order:

$$\tilde{\boldsymbol{y}}_\tau \sim q_\tau(\cdot \mid \boldsymbol{y}), \qquad \tilde{\boldsymbol{y}}_0 = \varepsilon, \qquad \tilde{\boldsymbol{y}}_1 = \boldsymbol{y}. \tag{2.7}$$

We choose Edit Flows over mask-based discrete diffusion alternatives such as MDLM [46] and SEDD [37] because deletion-based corruption yields valid token subsequences at every noise level, preserving compatibility with multiple frozen pretrained text encoders (Section 3.2). Thus, τ controls how much of the original sequence remains: at $\tau = 0$ only the $\langle \texttt{EOS} \rangle$ anchor is present, and at $\tau = 1$ the full sequence is recovered. For example, for $\boldsymbol{y} = \langle \texttt{A} \rangle \langle \texttt{black} \rangle \langle \texttt{cat} \rangle \langle \texttt{EOS} \rangle$, one possible corruption at $\tau = 0.5$ is $\tilde{\boldsymbol{y}}_\tau = \langle \texttt{cat} \rangle \langle \texttt{EOS} \rangle$, as illustrated in Figure 7b.

The reverse process reconstructs $\boldsymbol{y}$ by inserting missing tokens into the retained subsequence. For each retained token $\tilde{y}_\tau^i$ including $\langle \texttt{EOS} \rangle$, let A_i be the deleted tokens immediately to its left; in the

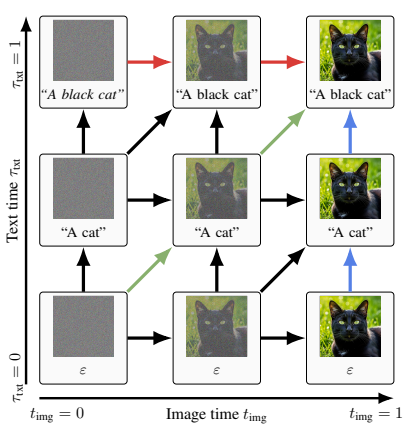


Figure 2: Dual-timestep space with image time t (x-axis) and text time τ (y-axis). All arrows, including the black ones, correspond to valid trajectories in the joint generative space. Colored trajectories highlight the canonical modes: ***text→image***, ***image→text***, and ***joint*** generation.

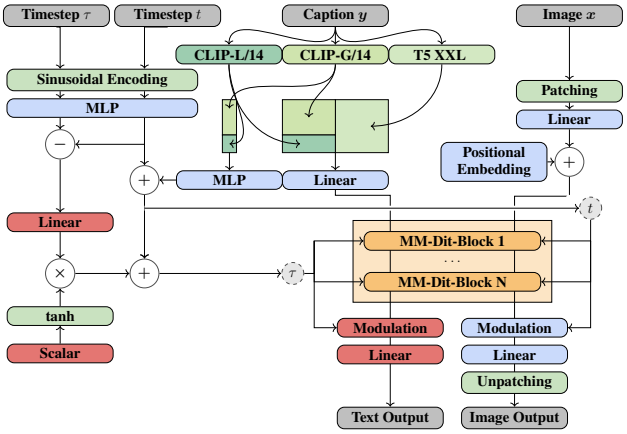


Figure 3: Overview of the multimodal DiT architecture adapted from Stable Diffusion 3. The model jointly processes noisy text tokens and image patches, along with their modality-specific timesteps (τ, t), to predict cross-modal flows. Blue blocks denote frozen pretrained components, yellow blocks indicate modules augmented with LoRA layers, red blocks mark newly added trainable components, and green blocks represent auxiliary operations.

example, $A_0 = \{\langle \texttt{A} \rangle, \langle \texttt{black} \rangle\}$ before $\langle \texttt{cat} \rangle$ and $A_1 = \emptyset$ before $\langle \texttt{EOS} \rangle$. The model parameterises the time-reversed CTMC with a nonnegative insertion rate $\lambda_\theta^i(\tilde{\boldsymbol{y}}_\tau, \tau)$ at each retained position and a categorical distribution $w_\theta^i(\cdot \mid \tilde{\boldsymbol{y}}_\tau, \tau)$ over inserted token identities.

Training combines a count loss and a token loss: $\mathcal{L}_{\text{txt}}(\theta) = \mathcal{L}_{\text{cnt}}(\theta) + \mathcal{L}_{\text{tok}}(\theta)$, defined as

$$\mathcal{L}_{\text{cnt}}(\theta) = \mathbb{E}_{\substack{\tau \sim \text{Unif}([0,1]) \\ \boldsymbol{y} \sim p_1}} \mathbb{E}_{\tilde{\boldsymbol{y}}_\tau \sim q_\tau(\cdot|\boldsymbol{y})} \left[\sum_i \lambda_\theta^i(\tilde{\boldsymbol{y}}_\tau, \tau) - |A_i| \cdot \log \lambda_\theta^i(\tilde{\boldsymbol{y}}_\tau, \tau) \right], \tag{2.8}$$

$$\mathcal{L}_{\text{tok}}(\theta) = \mathbb{E}_{\substack{\tau \sim \text{Unif}([0,1]) \\ \boldsymbol{y} \sim p_1}} \mathbb{E}_{\tilde{\boldsymbol{y}}_\tau \sim q_\tau(\cdot|\boldsymbol{y})} \left[- \sum_i \sum_{a \in A_i} \log w_\theta^i(a \mid \tilde{\boldsymbol{y}}_\tau, \tau) \right]. \tag{2.9}$$

The first term is the Poisson negative log-likelihood for the number of insertions, up to constants independent of θ, and the second term is the cross-entropy loss for the inserted token identities. Together, they define a reverse-time insertion process that transports ε to natural text.

3 Methodology

We describe how to upgrade a pretrained rectified-flow text-to-image model into a bidirectional vision–language generator: the native-space joint formulation, parameter-efficient backbone extension, dual-timestep objective and stabilizers, and inference interface.

3.1 Native-Space Multimodal Extension

Given a pretrained text-to-image model, we add text-generation capability while preserving the original image prior. We retain the pretrained continuous image process [35, 32] and introduce a discrete insertion process for text in the T5 tokenizer space, built on Edit Flows [16]. We choose Edit Flows over mask diffusion because their deletion-based forward process keeps every intermediate state $\tilde{\boldsymbol{y}}_\tau$ a valid token subsequence that can be decoded and re-tokenized for SD3's and FLUX's auxiliary CLIP encoders, whereas mask tokens fall outside their frozen vocabularies.

Let $(\boldsymbol{x}, \boldsymbol{y}) \sim p_1$ denote paired image–text data. Images are corrupted by $q_t^x(\boldsymbol{x}_t \mid \boldsymbol{x})$ and reconstructed with an image velocity head v_θ. Text is corrupted by $q_\tau^y(\tilde{\boldsymbol{y}}_\tau \mid \boldsymbol{y})$ via token deletion and reconstructed with lightweight heads for insertion counts and token identities. A single shared backbone jointly processes $(\boldsymbol{x}_t, t, \tilde{\boldsymbol{y}}_\tau, \tau)$, making the two modalities mutually informative at every forward pass.

3.2 Parameter-Efficient Extension of the Backbone

We freeze the backbone and train only: (i) a conditioning pathway for the new text timestep τ, (ii) text heads for insertion counts and token identities, and (iii) LoRA adapters in transformer layers [21]. Figure 3 illustrates the modifications to the SD3 MMDiT backbone. With LoRA rank $r = 32$, this yields $\sim$136M trainable parameters for SD3 ($\approx 5\%$ of total) and $\sim$298M for FLUX.1 ($\approx 2.5\%$).

SD3 conditions on three frozen encoders (T5-XXL and two CLIP variants) using both token-level and pooled embeddings. We perform discrete text diffusion entirely in the T5 tokenizer space, then bridge to the auxiliary encoders by decoding the partially deleted sequence $\tilde{\boldsymbol{y}}_\tau$ to a string and re-tokenizing it for each CLIP encoder, which is precisely possible because our chosen deletion produces valid token fragments rather than out-of-vocabulary mask symbols. Details are given in Section G.

3.3 Dual-Timestep Training Objective

Image and text are corrupted independently according to modality-specific timesteps t and τ, whose joint distribution is specified by a training schedule π. Given the partially corrupted pair $(\boldsymbol{x}_t, \tilde{\boldsymbol{y}}_\tau)$, the model predicts both the image velocity and the missing text insertions:

$$\mathcal{L}(\theta) = \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})\sim p_1}\mathbb{E}_{(t,\tau)\sim\pi}\mathbb{E}_{\substack{\boldsymbol{x}_t\sim q_t^x(\cdot|\boldsymbol{x}) \\ \tilde{\boldsymbol{y}}_\tau\sim q_\tau^y(\cdot|\boldsymbol{y})}}[\mathcal{L}_{\text{img}}(\theta;\, \boldsymbol{x}_t, t, \tilde{\boldsymbol{y}}_\tau, \tau) + \lambda_{\text{txt}}\, \mathcal{L}_{\text{txt}}(\theta;\, \tilde{\boldsymbol{y}}_\tau, \tau, \boldsymbol{x}_t, t)], \quad (3.1)$$

where $\mathcal{L}_{\text{img}}$ and $\mathcal{L}_{\text{txt}}$ are the rectified-flow and insertion losses from Equations (2.4) and (2.8), and λ_{txt} controls the relative contribution of the text objective.

Time coupling. The schedule $\pi(t, \tau)$ determines which jointly corrupted states are seen during training. The *alternating-clean* baseline

$$\pi_{\text{ac}}: \quad \text{w.p. } \tfrac{1}{2},\ t \sim \text{Unif}([0,1]),\ \tau = 1; \qquad \text{w.p. } \tfrac{1}{2},\ t = 1,\ \tau \sim \text{Unif}([0,1]), \quad (3.2)$$

matches Dual Diffusion [31] but supervises only the two endpoint-conditioned tasks, leaving the model without signal on intermediate joint states; this can push the two conditioning pathways toward orthogonal representations that impede cross-modal alignment. Our default

$$\pi_{\text{ind}}: \quad (t, \tau) \sim \text{Unif}([0,1]^2), \quad (3.3)$$

exposes the model to all combinations of partial image and partial text, directly training cross-modal correspondence across the full (t, τ) space.

3.4 Stabilization

Adding text generation to a pretrained image model creates two challenges: gradient scale mismatch, since the image branch is near convergence while text heads are randomly initialized; and image-prior degradation from fine-tuning outside the original data distribution. We address each with a stabilizer.

Adaptive gradient balancing. The two objectives induce gradients at different scales: the image branch is pretrained and uses an MSE velocity loss over high-dimensional latents, while the newly initialized text heads use count and cross-entropy losses with larger output-level gradients (Section E). Text updates can therefore dominate shared-parameter training early on. We periodically estimate the relative gradient scale on shared parameters and update λ_{txt} by EMA:

$$r = \frac{\|\nabla_{\theta_{\text{sh}}} \mathcal{L}_{\text{img}}\|_2}{\|\nabla_{\theta_{\text{sh}}} \mathcal{L}_{\text{txt}}\|_2 + \epsilon}, \qquad \lambda_{\text{txt}} \leftarrow \beta\lambda_{\text{txt}} + (1-\beta)\, r, \quad (3.4)$$

with $\beta = 0.99$ and $\epsilon = 10^{-8}$, adding negligible overhead ($< 0.1\%$ runtime) and removing the need to hand-tune a fixed weight per backbone.

Teacher matching. To preserve the pretrained image prior when fine-tuning on data outside the original pretraining distribution, we replace the velocity regression with a *teacher-matching* objective:

$$\mathcal{L}_{\text{img}}(\theta) = \|\text{sg}[v_{\text{base}}(\boldsymbol{x}_t, t, \bar{\boldsymbol{y}})] - v_\theta(\boldsymbol{x}_t, t, \tilde{\boldsymbol{y}}_\tau, \tau)\|_2^2, \quad (3.5)$$

where $\text{sg}[\cdot]$ stops gradients and $\bar{\boldsymbol{y}}$ is the teacher conditioning (clean text $\bar{\boldsymbol{y}} = \boldsymbol{y}$ or same-noise text $\bar{\boldsymbol{y}} = \tilde{\boldsymbol{y}}_\tau$). Crucially, the teacher v_{base} is obtained by simply disabling the LoRA layers of the same backbone, so no additional frozen copy needs to be held in memory.

3.5 Inference Modes

Trajectory selection in the time space. Decoupling t and τ turns inference into trajectory selection in a two-dimensional generative space; see Figure 2. Fixing $\tau = 1$ recovers text→image; fixing $t = 1$ gives image→text; evolving both jointly yields unconditional image–text sampling. Any trajectory through the (t, τ) square is valid, enabling flexible co-denoising without architectural change.

Partial-Text Generation for Downstream Tasks Restricting the insertion process to a target span while fixing all other tokens gives a simple, decoder-free interface for $(\text{image}, \text{text}) \to \text{text}$ tasks: for VQA, we condition on the image and question and generate only the answer span, extending the same backbone and inference procedure to structured understanding without any additional components.

4 Related Work

Unified multimodal generation. Earlier diffusion-based systems such as Versatile Diffusion and UniDiffuser showed that one diffusion-style model can support text-to-image, image-to-text, and joint or marginal generation through multi-flow designs or modality-specific timesteps [55, 5]. Later unified models explored hybrid or tokenized alternatives: Transfusion and Show-o combine diffusion with autoregressive text modeling, while Chameleon and Janus use tokenized visual representations in shared transformers [61, 54, 50, 53]. Recent non-autoregressive systems such as UniDisc and Muddit move closer to unified image–text diffusion, with Muddit also reusing a pretrained text-to-image backbone [49, 48]. Most of these approaches nevertheless require large-scale pretraining or substantial retraining, whereas we target a lightweight adaptation of existing text-to-image flow models. Specialized vision–language models such as Flamingo, BLIP, and LLaVA achieve strong multimodal understanding through large-scale autoregressive pretraining [2, 28, 27]; our goal is bidirectional generation from a single lightweight-adapted model, not maximal understanding performance alone.

Table 1: Trainable-component variants trained for 100k steps; text-conditioning variants for 200k steps. Metrics averaged over 5k validation examples.

Trainable components

Variant	CIDEr ↑
LoRA only	**40.1**
Refiner only	0.0
LoRA + Refiner	19.9

Text-conditioning pathways

Metric	Only T5	All Enc.
CMMD ↓	0.50	**0.11**
FID ↓	44.55	**34.25**
CIDEr ↑	63.96	**65.42**
CLIP ↑	**0.26**	0.25

Flow-based multimodal models. OmniFlow extends rectified flow to any-to-any multimodal generation with an MMDiT architecture and modular modality streams [30]. OneFlow combines image Flow Matching with insertion-based Edit Flows for concurrent interleaved generation [41]. FUDOKI formulates both modalities with discrete flow matching, while FlowTok maps text and images into a shared compact token space [52, 17]. JanusFlow harmonizes autoregressive text generation with rectified flow for images inside a unified transformer with decoupled visual encoders [38]; Janus-Pro scales this approach further [9]. The closest prior work, Dual Diffusion, similarly couples continuous image flow with masked text diffusion in an MM-DiT backbone, but trains endpoint-conditioned tasks rather than the full (t, τ) joint space, uses mask diffusion that yields non-decodable text states, and requires large-scale end-to-end retraining [31]. In contrast, we keep images in the pretrained rectified-flow latent space and text in a deletion-based insertion process whose corrupted states remain valid strings—preserving frozen multi-encoder compatibility and allowing the backbone to recover its pretrained image prior exactly—while requiring only LoRA adapters and lightweight text heads trainable on two RTX A5000 GPUs.

Diffusion-language-model multimodal models. A complementary line starts from diffusion language models. LaViDa, Dimple, and LLaDA-V attach visual encoders to masked or discrete diffusion language backbones, mainly targeting multimodal understanding [29, 59, 57]. More recent models such as MMaDA and LLaDA2.0-Uni extend this paradigm toward unified understanding, reasoning, image generation, and editing [56, 1]. Our direction is the reverse: we start from a strong pretrained text-to-image flow model and add a lightweight text-generation process.

5 Experiments

Our experiments address five questions: (i) which training ingredients are necessary for stable, resource-efficient adaptation (§5.2); (ii) how the adapted model compares to the closest prior work,

Dual Diffusion (§5.3); (iii) whether the same interface supports downstream understanding (§5.4); (iv) whether the recipe transfers across backbones (§5.5)s; and (v) which new inference modes are enabled by the dual-timestep formulation (§5.6).

5.1 Experimental Setup

We evaluate SD3-M [12] as the primary backbone and FLUX.1-dev [25] as a transfer setting. Models are fine-tuned on a fixed LAION-Aesthetic subset annotated with Moondream captions [47]. SD3 experiments use 256×256 resolution on two RTX A5000 GPUs; FLUX.1-dev experiments use three GPUs with manual sharding. We evaluate text→image with FID [19] and CMMD [23], computed against the frozen base SD3 model on a 1.9k-prompt suite curated for this work that spans natural, compositional, and dense long-form prompts. We evaluate image→text with CIDEr [51] and BERTScore-F1 [60] against held-out Moondream captions on a 5k validation split, joint generation with CLIP [44] score, and downstream VQA using official benchmark protocols. Further dataset, prompt, and implementation details are provided in Section B.

5.2 Adaptation Recipe Ablations

We isolate the ingredients needed to stably uplift a pretrained text-to-image backbone under limited compute, ablating each in turn.

A1: Trainable parameters and text-conditioning pathways. SD3 uses three frozen text encoders with distinct tokenizers, whereas our discrete text process is defined in a single T5 token space. We therefore diffuse T5 tokens, decode the partially corrupted sequence, and re-tokenize it for the remaining encoders.

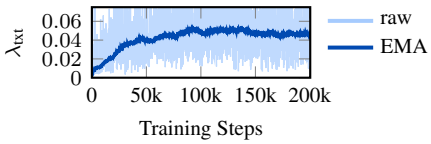


Figure 4: Evolution of the adaptive text-loss weight λ_{txt}. Thin: raw ratio estimate; thick: EMA used in training.

Text-conditioning. Although diffusion is performed only in T5 token space, conditioning all frozen SD3 text encoders substantially improves text→image retention while leaving image→text quality nearly unchanged. We therefore retain the full conditioning stack.

Trainable parameters. Full fine-tuning of SD3 is infeasible under our compute budget, so we restrict adaptation to lightweight modules and ask whether additional trainable capacity beyond LoRA is warranted. Table 1 compares LoRA-only updates to the shared backbone against an extended variant that adds a text refiner, a stack of single-stream DiT blocks placed before the text heads. The refiner does not improve image→text quality at matched parameter budget, so we adopt LoRA-only adaptation, resulting in updating $\sim$136M parameters in total ($\sim$5% of SD3-M).

A2: Loss-balance stabilization. The image loss is an MSE regression objective while the text loss combines count NLL and token cross-entropy. In our setup, text gradients are roughly $25\times$ larger than image gradients on shared parameters, but this ratio is backbone-specific and difficult to estimate a priori, so a fixed weight λ_{txt} would require a per-backbone hyperparameter sweep to avoid divergence or modality collapse. The EMA update from Section 3.4 eliminates this hyperparameter by adapting λ_{txt} from observed gradient ratios. As shown in Figure 4, the raw ratio is highly noisy, but its EMA settles on a stable value near 0.04 for SD3 at negligible overhead ($< 0.1\%$ runtime), yielding a value we found difficult to identify by inspection alone.

Table 2: Image-prior preservation: direct flow matching (FM), same-noise teacher matching (Teacher-SN), and clean-text teacher matching (Teacher-CT).

Variant	CMMD↓	FID↓
FM	0.579	63.46
Teacher-SN	**0.084**	**30.38**
Teacher-CT	0.310	39.49

A3: Preserving the pretrained image prior. Adding text generation can degrade the pretrained image prior. We compare three image objectives: direct flow matching against ground-truth latents (**FM**); same-noise teacher matching (**Teacher-SN**), where the frozen text-to-image teacher denoises the same noisy latent as the student conditioned on the corrupted text; and clean-text teacher matching (**Teacher-CT**), where the teacher conditions on the clean caption while the student conditions on the corrupted text. Table 2 shows that Teacher-SN gives the best fidelity–retention trade-off, reducing FID from 63.46 to 30.38 and CMMD from 0.579 to 0.084. We adopt Teacher-SN in all subsequent experiments.

Table 4: Cross-modal generation on SD3 with matched data, LoRA rank, and training setup. FID and CMMD are computed against the frozen base SD3 model; CIDEr and BERT-F1 are computed against held-out Moondream captions on the validation split; full details in Table 9.

	Text→Image		Image→Text	
Method	**FID↓**	**CMMD↓**	**CIDEr↑**	**BERT-F1↑**
DualDiff.-LoRA	62.65	0.870	2.0	−0.27
Ours, step-match	**29.32**	**0.021**	13.4	0.02
Ours, time-match	31.57	0.121	**99.4**	**0.44**

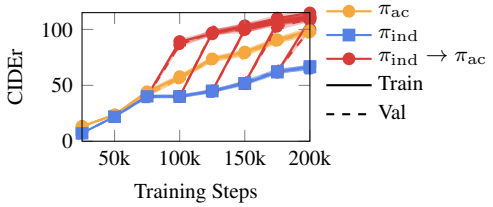


Figure 5: CIDEr for alternating-clean (π_{ac}), mixed-corruption (π_{ind}), and five independent runs that switch from π_{ind} to π_{ac} at 75k, 100k, 125k, 150k, and 175k steps. Mean over a 5k held-out split; error bars show 95% CIs.

A4: Which corruption schedule best supports both correspondence and deployment? We compare two corruption schedules: alternating-clean (π_{ac}), which keeps one modality clean and matches the endpoint-conditioned setting used at deployment for pure captioning, and mixed-corruption (π_{ind}), which samples $(t, \tau) \sim \mathrm{Unif}([0,1]^2)$ and exposes the model to jointly corrupted image–text states. In isolation, π_{ac} outperforms π_{ind} on captioning (Figure 5), which we attribute to a closer match between training and deployment noise distributions for that task. We hypothesize, however, that π_{ind} provides broader joint supervision that is useful as a pretraining stage prior to specialization. To test this, we run five independent fine-tuning experiments that switch from π_{ind} to π_{ac} at $\{75\text{k}, 100\text{k}, 125\text{k}, 150\text{k}, 175\text{k}\}$ steps; all five converge to a CIDEr score that surpasses the pure-π_{ac} run, reaching the pure-π_{ac} final value in roughly half the training steps. We therefore adopt mixed-corruption pretraining followed by alternating-clean refinement, using 100k π_{ind} steps followed by 50k π_{ac} steps (total wall-clock time under 24 h) in all subsequent SD3 experiments. For convergence analysis, runs were continued to 200k total steps.

5.3 Comparison to Dual Diffusion

Dual Diffusion is originally trained end-to-end at large scale, which would confound architectural and training-budget effects. To isolate the modeling contribution, we construct a matched baseline (*DualDiff.-LoRA*) by re-implementing its architecture on the same SD3 backbone with LoRA at the same rank used by FullFlow ($r = 32$), identical data, and the same optimizer; trainable parameter counts are nearly identical (136.8M vs. 136.1M). The two methods differ only in their cross-modal interface: DualDiff.-LoRA retains Dual Diffusion's masked text-diffusion pathway and partially trains text-conditioning components, whereas FullFlow operates on valid partially deleted strings via the tokenizer-space Edit Flow with all text encoders frozen, enabling same-noise teacher matching for image-prior retention. Since FullFlow is $\sim 8\times$ faster per step, we report *step-matched* and *time-matched* configurations.

Table 3: Downstream captioning and VQA for unified models. Grouped by paradigm: AR/hybrids (top), diffusion (middle), and ours (bottom). Parentheses denote training resolution. COCO reports CIDEr; VQAv2, VizWiz, and OKVQA report accuracy.

Model	**Trainable Params**	**COCO↑**	**VQAv2↑**	**VizWiz↑**	**OKVQA↑**
CM3Leon [58]	7B	61.6	47.6	37.6	23.8
Chameleon [50]	7B	18.0	–	–	–
LWM [33]	7B	–	55.8	11.6	–
Show-O (256) [54]	1.3B	–	64.7	–	–
Show-O (512) [54]	1.3B	–	69.4	–	–
Transfusion [61]	7B	29.0	–	–	–
DualDiff (256) [31]	2B	–	59.5	19.4	28.5
DualDiff (512) [31]	2B	56.2	60.1	29.9	25.3
FullFlow (Ours)	**130M**	54.93	56.5	50.7	23.3

Conditional generation. Table 4 reports cross-modal generation against DualDiff.-LoRA. We report step- and time-matched FullFlow configurations to isolate complementary effects: per-update modeling quality and practical advantage at a fixed compute budget. At matched wall-clock, FullFlow reduces FID from 62.65 to 31.57 and CMMD from 0.870 to 0.121, and improves CIDEr from 2.0 to 99.4 and BERTScore-F1 from -0.27 to 0.44. Step-matched gains move in the same direction with a smaller image→text margin (CIDEr 13.4), indicating that both architecture and per-step efficiency contribute to the improvement. Qualitative examples are provided in Figures 15, 16 and 18.

Efficiency. The matched-LoRA setting also exposes the computational impact. DualDiff.-LoRA backpropagates through its masked text-diffusion pathway and partially trains text-conditioning components, requiring ~ 84 GB peak VRAM. By keeping the text encoders fully frozen and operating

on valid partially deleted strings, FullFlow runs at $\sim$38 GB and raises throughput from 0.25 to 1.96 steps/s, fitting the full training on two RTX A5000 GPUs.

5.4 Downstream Understanding

We test whether the same text-generation interface supports downstream multimodal understanding. Table 3 positions FullFlow against two families: autoregressive and AR–diffusion hybrids (top), and fully diffusion-based models (middle). At 10–50$\times$ fewer trainable parameters than every baseline and at half the resolution of the strongest competitor, FullFlow attains the highest VizWiz accuracy in the table (50.7); on MS-COCO captioning it is competitive at 256 resolution (54.9 CIDEr) with DualDiff at 512 resolution (56.2) and the 7B CM3Leon (61.6), and remains close on VQAv2 (56.5, vs. 60.1 for DualDiff(512) and 69.4 for the larger Show-O). The weakest result is OKVQA (23.3, $\sim$18% below DualDiff(256)), which we attribute to that benchmark's reliance on external world knowledge that scales with pretraining-data scale: FullFlow is fine-tuned on a small LAION subset, whereas DualDiff is trained on multiple orders of magnitude more data. Overall, a fully diffusion-based interface matches autoregressive and hybrid unified models at a small fraction of their size, supporting a single diffusion process across both modalities; further comparison in Table 10.

5.5 Transfer to FLUX.1-dev.

To verify the recipe is not specific to SD3, we apply it to FLUX.1-dev with minimal architecture-specific changes, sharding the 22 GB model across three RTX A5000 GPUs at 0.49 steps/s. Following the two-stage schedule of Section 5.2, 40k π_{ind} steps reach CIDEr 57.5 and a further 20k π_{ac} steps raise it to 73.6, while the text$\rightarrow$image prior holds at FID 24.7 / CMMD 0.014; total wall-clock training is 36 h. Qualitative samples are provided in Figures 17 and 19.

5.6 Joint generation.

The dual-timestep formulation also enables image and text to be sampled jointly rather than conditioning on one modality. We sweep trajectories by fixing image time to a linear schedule and setting text time to $\tau = t^{2^p}$, with p controlling which modality denoises earlier. Figure 6 shows the diagonal trajectory ($p = 0$) yields the highest CLIP score, indicating that balanced co-denoising gives the strongest image–text agreement; text-first trajectories remain competitive, while delaying text degrades alignment. Qualitative examples are in Figures 22 to 25.

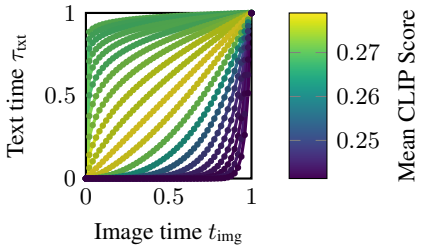


Figure 6: Joint generation over $\tau = t^{2^p}$ trajectories. Color shows mean CLIP over 1k samples.

6 Limitations and Conclusion

Limitations. We focused on a compute-feasible proof of concept rather than peak end-to-end performance. For this reason the main results in this work use data-, parameter-, and compute-matched comparisons against prior work (Section 5.3). We did not explore scaling of the method across: training at 512×512 and above, larger paired datasets, instruction tuning for VQA and dialogue (closing the gap to specialized AR VLMs), and porting the same uplift to other rectified-flow backbones. All these directions remain interesting future developments which are conceptually supported by our method, but would require more compute and time.

Conclusion. FullFlow upgrades pretrained text-to-image models into bidirectional image–text generators by operating in valid tokenizer space with the text-conditioning stack frozen, supporting text-to-image, captioning, and joint generation from a single adapted backbone. The fact that bidirectional capability emerges from such a thin adaptation suggests that pretrained text-to-image priors encode richer visual–semantic structure than commonly assumed, and that similar lightweight uplifts may extend to other larger backbones, where the compute savings would be even more impactful.

Broader impact. While broadening access to interactive image–text applications, our method may also amplify risks involving synthetic generation, biased captions, and dataset contamination.

References

[1] Inclusion AI, Tiwei Bie, Haoxing Chen, Tieyuan Chen, Zhenglin Cheng, Long Cui, Kai Gan, Zhicheng Huang, Zhenzhong Lan, Haoquan Li, et al. Llada2. 0-uni: Unifying multimodal understanding and generation with diffusion large language model. *arXiv preprint arXiv:2604.20796*, 2026.

[2] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems*, 35: 23716–23736, 2022.

[3] Anas Awadalla, Irena Gao, Josh Gardner, Jack Hessel, Yusuf Hanafy, Wanrong Zhu, Kalyani Marathe, Yonatan Bitton, Samir Gadre, Shiori Sagawa, et al. Openflamingo: An open-source framework for training large autoregressive vision-language models, 2023.

[4] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

[5] Fan Bao, Shen Nie, Kaiwen Xue, Chongxuan Li, Shi Pu, Yaole Wang, Gang Yue, Yue Cao, Hang Su, and Jun Zhu. One transformer fits all distributions in multi-modal diffusion at scale. In *International Conference on Machine Learning*, pages 1692–1717. PMLR, 2023.

[6] Eric Tillmann Bill, Enis Simsar, and Thomas Hofmann. Optimal control meets flow matching: A principled route to multi-subject fidelity, 2025.

[7] Andrew Campbell, Jason Yim, Regina Barzilay, Tom Rainforth, and Tommi Jaakkola. Generative flows on discrete state-spaces: Enabling multimodal flows with applications to protein co-design, 2024.

[8] Hila Chefer, Yuval Alaluf, Yael Vinker, Lior Wolf, and Daniel Cohen-Or. Attend-and-excite: Attention-based semantic guidance for text-to-image diffusion models, 2023.

[9] Xiaokang Chen, Zhiyu Wu, Xingchao Liu, Zizheng Pan, Wen Liu, Zhenda Xie, Xingkai Yu, and Chong Ruan. Janus-pro: Unified multimodal understanding and generation with data and model scaling. *arXiv preprint arXiv:2501.17811*, 2025.

[10] Zhe Chen, Weiyun Wang, Yue Cao, Yangzhou Liu, Zhangwei Gao, Erfei Cui, Jinguo Zhu, Shenglong Ye, Hao Tian, Zhaoyang Liu, Lixin Gu, Xuehui Wang, Qingyun Li, Yiming Ren, Zixuan Chen, Jiapeng Luo, Jiahao Wang, Tan Jiang, Bo Wang, Conghui He, Botian Shi, Xingcheng Zhang, Han Lv, Yi Wang, Wenqi Shao, Pei Chu, Zhongying Tu, Tong He, Zhiyong Wu, Huipeng Deng, Jiaye Ge, Kai Chen, Kaipeng Zhang, Limin Wang, Min Dou, Lewei Lu, Xizhou Zhu, Tong Lu, Dahua Lin, Yu Qiao, Jifeng Dai, and Wenhai Wang. Expanding Performance Boundaries of Open-Source Multimodal Models with Model, Data, and Test-Time Scaling, September 2025. URL `http://arxiv.org/abs/2412.05271`. arXiv:2412.05271 [cs].

[11] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale N. Fung, and Steven Hoi. InstructBLIP: Towards General-purpose Vision-Language Models with Instruction Tuning. *Advances in Neural Information Processing Systems*, 36:49250–49267, December 2023. URL `https://proceedings.neurips.cc/paper_files/paper/2023/hash/9a6a435e75419a836fe47ab6793623e6-Abstract-Conference.html`.

[12] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis, 2024.

[13] Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky TQ Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete flow matching, 2024.

[14] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. Making the v in vqa matter: Elevating the role of image understanding in visual question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6904–6913, 2017.

[15] Danna Gurari, Qing Li, Abigale J Stangl, Anhong Guo, Chi Lin, Kristen Grauman, Jiebo Luo, and Jeffrey P Bigham. Vizwiz grand challenge: Answering visual questions from blind people. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3608–3617, 2018.

[16] Marton Havasi, Brian Karrer, Itai Gat, and Ricky TQ Chen. Edit flows: Flow matching with edit operations, 2025.

[17] Ju He, Qihang Yu, Qihao Liu, and Liang-Chieh Chen. Flowtok: Flowing seamlessly across text and image tokens, 2025.

[18] Amir Hertz, Ron Mokady, Jay Tenenbaum, Kfir Aberman, Yael Pritch, and Daniel Cohen-Or. Prompt-to-prompt image editing with cross-attention control, 2023.

[19] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium, 2017.

[20] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020.

[21] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models., 2022.

[22] Kaiyi Huang, Kaiyue Sun, Enze Xie, Zhenguo Li, and Xihui Liu. T2i-compbench: A comprehensive benchmark for open-world compositional text-to-image generation, 2023.

[23] Sadeep Jayasumana, Srikumar Ramalingam, Andreas Veit, Daniel Glasner, Ayan Chakrabarti, and Sanjiv Kumar. Rethinking fid: Towards a better evaluation metric for image generation, 2024.

[24] Chen Jin, Ryutaro Tanno, Amrutha Saseendran, Tom Diethe, and Philip Alexander Teare. Diffusion instruction tuning. In *International Conference on Machine Learning*, pages 28097–28137. PMLR, 2025.

[25] Black Forest Labs. Flux. `https://github.com/black-forest-labs/flux`, 2024.

[26] Hugo Laurençon, Lucile Saulnier, Léo Tronchon, Stas Bekman, Amanpreet Singh, Anton Lozhkov, Thomas Wang, Siddharth Karamcheti, Alexander Rush, Douwe Kiela, et al. Obelics: An open web-scale filtered dataset of interleaved image-text documents, 2023.

[27] Feng Li, Renrui Zhang, Hao Zhang, Yuanhan Zhang, Bo Li, Wei Li, Zejun Ma, and Chunyuan Li. Llava-next-interleave: Tackling multi-image, video, and 3d in large multimodal models, 2024.

[28] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation, 2022.

[29] Shufan Li, Konstantinos Kallidromitis, Hritik Bansal, Akash Gokul, Yusuke Kato, Kazuki Kozuka, Jason Kuen, Zhe Lin, Kai-Wei Chang, and Aditya Grover. Lavida: A large diffusion language model for multimodal understanding, 2025.

[30] Shufan Li, Konstantinos Kallidromitis, Akash Gokul, Zichun Liao, Yusuke Kato, Kazuki Kozuka, and Aditya Grover. Omniflow: Any-to-any generation with multi-modal rectified flows, 2025.

[31] Zijie Li, Henry Li, Yichun Shi, Amir Barati Farimani, Yuval Kluger, Linjie Yang, and Peng Wang. Dual diffusion for unified image generation and understanding, 2025.

[32] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling, 2023.

[33] Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. World model on million-length video and language with blockwise ringattention, 2025.

[34] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.

[35] Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data with rectified flow, 2023.

[36] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[37] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution, June 2024. URL http://arxiv.org/abs/2310.16834. arXiv:2310.16834 [stat].

[38] Yiyang Ma, Xingchao Liu, Xiaokang Chen, Wen Liu, Chengyue Wu, Zhiyu Wu, Zizheng Pan, Zhenda Xie, Haowei Zhang, Xingkai Yu, et al. Janusflow: Harmonizing autoregression and rectified flow for unified multimodal understanding and generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7739–7751, 2025.

[39] Kenneth Marino, Mohammad Rastegari, Ali Farhadi, and Roozbeh Mottaghi. Ok-vqa: A visual question answering benchmark requiring external knowledge. In *Proceedings of the IEEE/cvf conference on computer vision and pattern recognition*, pages 3195–3204, 2019.

[40] Tuna Han Salih Meral, Enis Simsar, Federico Tombari, and Pinar Yanardag. Conform: Contrast is all you need for high-fidelity text-to-image diffusion models, 2024.

[41] John Nguyen, Marton Havasi, Tariq Berrada, Luke Zettlemoyer, and Ricky T. Q. Chen. OneFlow: Concurrent Mixed-Modal and Interleaved Generation with Edit Flows, October 2025. URL http://arxiv.org/abs/2510.03506. arXiv:2510.03506 [cs].

[42] Gaurav Parmar, Richard Zhang, and Jun-Yan Zhu. On aliased resizing and surprising subtleties in gan evaluation. In *CVPR*, 2022.

[43] William Peebles and Saining Xie. Scalable diffusion models with transformers, 2023.

[44] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision, 2021.

[45] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models, June 2022.

[46] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and Effective Masked Diffusion Language Models, November 2024. URL http://arxiv.org/abs/2406.07524. arXiv:2406.07524 [cs].

[47] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, Patrick Schramowski, Srivatsa Kundurthy, Katherine Crowson, Ludwig Schmidt, Robert Kaczmarczyk, and Jenia Jitsev. LAION-5B: An open large-scale dataset for training next generation image-text models, October 2022. URL http://arxiv.org/abs/2210.08402. arXiv:2210.08402 [cs].

[48] Qingyu Shi, Jinbin Bai, Zhuoran Zhao, Wenhao Chai, Kaidong Yu, Jianzong Wu, Shuangyong Song, Yunhai Tong, Xiangtai Li, Xuelong Li, et al. Muddit: Liberating generation beyond text-to-image with a unified discrete diffusion model, 2025.

[49] Alexander Swerdlow, Mihir Prabhudesai, Siddharth Gandhi, Deepak Pathak, and Katerina Fragkiadaki. Unified multimodal discrete diffusion, 2025.

[50] Chameleon Team. Chameleon: Mixed-Modal Early-Fusion Foundation Models, March 2025. URL http://arxiv.org/abs/2405.09818. arXiv:2405.09818 [cs].

[51] Ramakrishna Vedantam, C Lawrence Zitnick, and Devi Parikh. Cider: Consensus-based image description evaluation, 2015.

[52] Jin Wang, Yao Lai, Aoxue Li, Shifeng Zhang, Jiacheng Sun, Ning Kang, Chengyue Wu, Zhenguo Li, and Ping Luo. Fudoki: Discrete flow-based unified understanding and generation via kinetic-optimal velocities, 2025.

[53] Chengyue Wu, Xiaokang Chen, Zhiyu Wu, Yiyang Ma, Xingchao Liu, Zizheng Pan, Wen Liu, Zhenda Xie, Xingkai Yu, Chong Ruan, and Ping Luo. Janus: Decoupling Visual Encoding for Unified Multimodal Understanding and Generation, October 2024. URL http://arxiv.org/abs/2410.13848. arXiv:2410.13848 [cs].

[54] Jinheng Xie, Weijia Mao, Zechen Bai, David Junhao Zhang, Weihao Wang, Kevin Qinghong Lin, Yuchao Gu, Zhijie Chen, Zhenheng Yang, and Mike Zheng Shou. Show-o: One single transformer to unify multimodal understanding and generation, 2025. URL https://openreview.net/forum?id=o6Ynz6OIQ6.

[55] Xingqian Xu, Zhangyang Wang, Eric Zhang, Kai Wang, and Humphrey Shi. Versatile Diffusion: Text, Images and Variations All in One Diffusion Model, January 2024. URL http://arxiv.org/abs/2211.08332. arXiv:2211.08332 [cs].

[56] Ling Yang, Ye Tian, Bowen Li, Xinchen Zhang, Ke Shen, Yunhai Tong, and Mengdi Wang. MMaDA: Multimodal large diffusion language models, 2026. URL https://openreview.net/forum?id=wczmXLuLGd.

[57] Zebin You, Shen Nie, Xiaolu Zhang, Jun Hu, Jun Zhou, Zhiwu Lu, Ji-Rong Wen, and Chongxuan Li. Llada-v: Large language diffusion models with visual instruction tuning, 2025.

[58] Lili Yu, Bowen Shi, Ramakanth Pasunuru, Benjamin Muller, Olga Golovneva, Tianlu Wang, Arun Babu, Binh Tang, Brian Karrer, Shelly Sheynin, Candace Ross, Adam Polyak, Russell Howes, Vasu Sharma, Puxin Xu, Hovhannes Tamoyan, Oron Ashual, Uriel Singer, Shang-Wen Li, Susan Zhang, Richard James, Gargi Ghosh, Yaniv Taigman, Maryam Fazel-Zarandi, Asli Celikyilmaz, Luke Zettlemoyer, and Armen Aghajanyan. Scaling Autoregressive Multi-Modal Models: Pretraining and Instruction Tuning, September 2023. URL http://arxiv.org/abs/2309.02591. arXiv:2309.02591 [cs].

[59] Runpeng Yu, Xinyin Ma, and Xinchao Wang. Dimple: Discrete Diffusion Multimodal Large Language Model with Parallel Decoding, May 2025. URL http://arxiv.org/abs/2505.16990. arXiv:2505.16990 [cs].

[60] Tianyi Zhang*, Varsha Kishore*, Felix Wu*, Kilian Q. Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert, 2020. URL https://openreview.net/forum?id=SkeHuCVFDr.

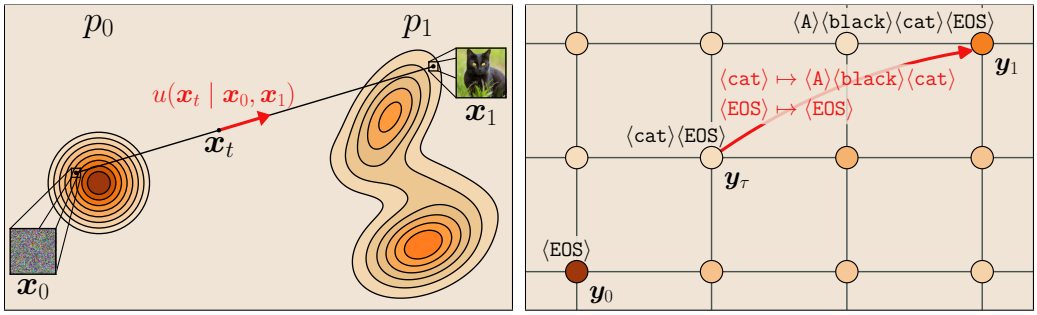
[61] Chunting Zhou, LILI YU, Arun Babu, Kushal Tirumala, Michihiro Yasunaga, Leonid Shamis, Jacob Kahn, Xuezhe Ma, Luke Zettlemoyer, and Omer Levy. Transfusion: Predict the next token and diffuse images with one multi-modal model, 2025. URL https://openreview.net/forum?id=SI2hI0frk6.

[62] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *arXiv preprint arXiv:2304.10592*, 2023.

Appendix

A Flow Matching: Similarity Between Continuous and Discrete

Despite their apparent differences, continuous rectified flow and discrete Edit Flows instantiate the same fundamental principle: learn a model to predict local transport updates along a path from a base distribution to the data distribution. In continuous flow matching, we learn a velocity field that describes how samples move continuously through latent space. In discrete flow matching applied to text, we instead learn to predict insertion discrete token placements that progressively transform an empty sequence into the target sequence. Both formulations share the same path-based training objective: at each point along the trajectory, the model is trained to make the locally-optimal next move (either a velocity in continuous space or a jump in discrete space) to reach the data. This unified perspective on flow matching enables us to train image and text processes with a shared modeling philosophy while respecting the geometric differences between continuous image latents and discrete token sequences. Figure 7 illustrates this conceptual connection.



(a) Continuous flow matching

(b) Discrete flow via jumps

Figure 7: Continuous rectified flow (left) and discrete Edit Flows (right) instantiate the same path-based principle: learn local transport from a base distribution to data. In continuous space the supervised target is a velocity; in discrete space it is an insertion jump. The red arrow/token marks the quantity the model is trained to predict.

B Training Details

B.1 Dataset Preparation and Pre-Encoding

We train on a subset of the LAION-COCO-Aesthetic collection [47], a curated split of LAION-2B-EN filtered for aesthetic quality scores and paired with semantically aligned captions. Before training we apply a mild aspect-ratio filter (retaining images with width/height ratio below 1.4 in both orientations) to avoid extreme crops, yielding a training set of $\sim$280,000 image–caption pairs.

Images are pre-encoded offline with the SD3 VAE to avoid redundant encoder passes during training. Each image is resized to 256×256 with bicubic interpolation and center-cropped, normalized to $[-1, 1]$, then encoded to a latent $\boldsymbol{x}_1 \in \mathbb{R}^{16\times32\times32}$ via

$$\boldsymbol{x}_1 = \big(\text{VAE-Enc}(\text{img}) - s_{\text{shift}}\big) \cdot s_{\text{scale}}, \tag{B.1}$$

where s_{shift} and s_{scale} are the SD3 VAE shift and scaling factors. The encoded latents and raw caption strings are saved to disk; the VAE is not loaded during training. We visualize the caption-length distribution of the dataset in Figure 8.

B.2 Text-to-Image Evaluation Prompt Suite

For text→image evaluation, we use a fixed prompt suite rather than a held-out LAION split. The goal is to better approximate open-ended generation, where prompts vary in length, compositional complexity, and descriptive density. We therefore combine three complementary sources.

First, we sample natural image captions from MS-COCO, stratified by a simple regex word-token count: short captions contain 4–8 tokens, medium captions contain 9–17 tokens, and long captions

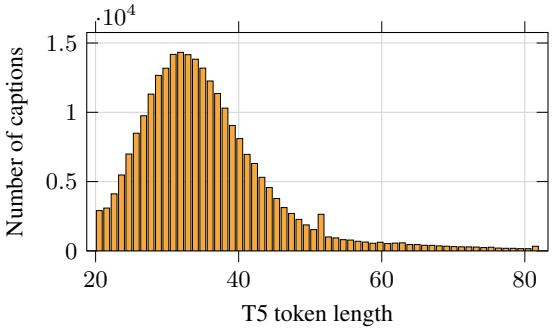


Figure 8: Distribution of caption lengths in the training set, measured in T5 tokens after the LAION-COCO-Aesthetic filtering described in Section B. For readability the histogram drops the lowest and highest 1% of caption lengths and uses 60 bins; this distribution motivates our 77-token sequence cap.

contain 18 or more tokens; captions with fewer than 4 tokens are discarded. We sample 300 captions from each length bucket. Second, we sample compositional prompts from T2I-CompBench [22], using 100 prompts from each of eight categories: color, shape, texture, spatial, 3D spatial, non-spatial, numeracy, and complex. Third, we include 200 dense long-form prompts from DrawBench-upsampled, stratified across its available categories when possible. This yields a nominal 1.9k-prompt suite before cross-source deduplication.

All prompts are whitespace-normalized, lowercased for duplicate detection, and deduplicated across sources. We use a fixed random seed for sampling and keep the same prompt suite for all methods.

B.3 Resolution: Why 256×256?

SD3 includes 256×256 images in its multi-resolution pretraining curriculum [12]. We therefore evaluate at 256×256, a resolution already covered by the pretrained backbone, while keeping the image-token sequence length small enough for our two-GPU setting. At this resolution, the SD3 VAE encodes images into 32×32 spatial latents, which SD3's patch-embed layer (2×2 patches) reduces to $n = 256$ image tokens. The joint sequence length—image tokens plus three text encoder outputs of length 77 each, is therefore manageable within the 2×24 GB GPU budget.

Compared to higher resolutions, a 512×512 input quadruples the image token count to $1{,}024$, increasing the full-sequence self-attention cost by roughly $16\times$, which is prohibitive for our hardware setup. Compared to 128×128, the VAE compression is less severe at 256×256 and perceptual quality is substantially better, making image-generation evaluation more meaningful. The 256×256 regime therefore gives the best compute–quality trade-off available on two RTX A5000 GPUs.

B.4 Optimizer and Learning-Rate Schedule

We use AdamW [36] with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, and *two parameter groups* with separate learning rates and regularization:

- **LoRA adapters and timestep processors** (text timestep τ extra linear layer and scalar.): lr $= 10^{-4}$, weight decay $= 0$.
- **Final AdaLN text modulation and text heads** and other newly introduced weights not covered by LoRA: lr $= 5 \times 10^{-4}$, weight decay $= 10^{-2}$.

A constant schedule with a $5{,}000$-step linear warmup is used; no learning-rate decay is applied. Gradients are clipped to a global ℓ_2 norm of 2.0 before each optimizer step.

B.5 LoRA Configuration

LoRA adapters [21] are attached to all linear projection layers inside the `transformer_blocks` of the MM-DiT backbone (query, key, value, output projections of both self-attention streams). New heads (text-conditioning layers and text output heads) are excluded from LoRA and trained directly. The LoRA configuration is:

Hyperparameter	Value
Rank r	32
Scaling α	32
Dropout	0.05
Target modules	All `nn.Linear` in `transformer_blocks`
Bias	none

With this configuration, the total trainable parameter count is $\sim$136M for SD3-M ($\approx 5\%$ of the full model) and $\sim$298M for FLUX.1 ($\approx 2.5\%$).

B.6 Zero-Inflated Insertion Counts

Both backbones we adapt operate at a fixed sequence length (77 for SD3, 128 for FLUX) inherited from their pretrained text encoders. Under deletion-to-empty corruption with retention probability τ, the vast majority of retained positions have no deleted left-neighbours ($|A_i| = 0$), especially at high τ where most tokens are kept. The Poisson target on λ_θ^i is therefore strongly skewed toward zero, biasing the rate head and starving the model of gradient signal on the rare nonzero counts that actually drive insertion behaviour.

Following OneFlow [41], we factor the insertion-count distribution as a zero-inflated Poisson by adding a binary gating head $\pi_\theta^i(\tilde{\boldsymbol{y}}_\tau, \tau) \in [0, 1]$ that predicts $\mathbb{P}(|A_i| > 0)$, while λ_θ^i models the Poisson rate conditional on $|A_i| > 0$. The text loss Equation (2.8) then becomes

$$\mathcal{L}_{\text{txt}}(\theta) = \text{BCE}\big(\pi_\theta^i,\, \mathbb{1}\{|A_i| > 0\}\big) \;+\; \mathbb{1}\{|A_i| > 0\} \cdot \mathcal{L}_{\text{Poisson}}(\lambda_\theta^i; |A_i|) \;+\; \mathcal{L}_{\text{tok}}(w_\theta^i; A_i),$$

summed over retained positions i, with the Poisson and token-identity losses evaluated only on positions with at least one insertion. At inference, position i is kept empty with probability $1 - \pi_\theta^i$ and otherwise samples a count from λ_θ^i followed by token identities from w_θ^i. This decouples the abundant zero-class supervision from the rare nonzero counts and prevents the rate head from collapsing toward zero on long padded sequences.

B.7 Training Hyperparameters

Table 5 lists the training hyperparameters for the main FullFlow-SD3 run and the matched DualDiff.-LoRA baseline. The two runs use the same optimizer, batch size, learning-rate schedule, LoRA configuration, and training budget. They differ only in the corruption schedule and image-supervision target: FullFlow uses a mixed-to-alternating schedule, training with π_{ind} for the first 100k steps and then switching to π_{ac} via `ac_sched`, whereas DualDiff.-LoRA uses π_{ac} throughout. FullFlow also uses same-noise teacher matching (`teacher_matching=noisy`), while DualDiff.-LoRA uses the standard rectified-flow image objective.

B.8 DualDiff.-LoRA Baseline Reproduction

We re-implement the DualDiff.-LoRA baseline by directly using the official Dual Diffusion training pipeline[1], with two modifications to match our setup:

- *Data loader.* Replaced with our pre-encoded LAION-Aesthetic loader (Section B), so that the VAE is not held in VRAM and the dataset, captions, and resolution exactly match FullFlow.
- *Trainable parameters.* Instead of full backbone fine-tuning, we attach the same LoRA configuration as FullFlow (rank $r = 32$ on all linear layers in `transformer_blocks`; Section B) and additionally train the newly initialized text-conditioning modules and text heads.

All other Dual-Diffusion components, including the mask-noise schedule, text-head architecture, sampler, and DD-specific hyperparameters, are kept exactly as released. The optimizer, learning-rate schedule, batch size, and total number of training steps match the FullFlow training run (Table 5); the trainable parameter counts are also nearly matched, at $\sim$136.8M for DualDiff.-LoRA and $\sim$136.1M for FullFlow.

[1] `https://github.com/zijieli-Jlee/Dual-Diffusion`

Table 5: Training hyperparameters for the SD3-M experiments. Values apply to both FullFlow and the matched DualDiff.-LoRA baseline unless explicitly noted; the two runs differ only in the corruption schedule and the image-supervision target.

Hyperparameter	**Value**	**Note**
Hardware & precision		
GPUs	$2 \times$ RTX A5000 (24 GB)	DDP, NCCL backend
Mixed precision	BFloat16	Optimizer states in FP32
Gradient checkpointing	No	
Data & batching		
Training resolution	256×256	Pre-encoded VAE latents
Training set size	$\sim$280,000 image–caption pairs	LAION-COCO-Aesthetic
Global batch size	10	5 per GPU
Micro-batch size	5	1 accumulation step
Max sequence length	77	T5 tokenizer tokens
Optimisation		
Optimizer	AdamW	β_1=0.9, β_2=0.999, ε=10^{-8}
LoRA / head learning rate	1×10^{-4}	No weight decay
Base parameter learning rate	5×10^{-4}	Weight decay 10^{-2}
LR schedule	Constant + warmup	5,000 warmup steps
Gradient clip (ℓ_2)	2.0	
Training steps	200,000	
Random seed	42	
Adaptive gradient balancing		
Estimation frequency	Every 100 steps	On a micro-batch of size 3
EMA decay β	0.99	
Gradient ratio scale	5.0	Applied before EMA
Loss weights		
Image weight w_{img}	1.0	
Text weight w_{txt}	0.05	Scaled further by λ_{txt}
Teacher matching	noisy	(`Teacher-SN`)
EMA of model weights		
EMA decay	0.9995	Applied to trainable params
EMA dtype	FP32	

B.9 VQA Finetuning

VQA adaptation follows the partial-text generation interface described in Section 3.5: the image and the question tokens are held fixed, and the insertion process is restricted to the answer span. The model learns to denoise only the answer tokens, conditioned on the image and the full question.

Concretely, finetuning resumes from a pretrained FullFlow checkpoint and replaces the joint image–text loss with a text-only VQA loss. No image velocity supervision is applied during this stage ($w_{\text{img}} = 0$). The trainable parameters and LoRA configuration are identical to the pretraining stage.

Training runs for an additional $100{,}000$ steps on pre-encoded VQAv2 question–answer–image triples at 256×256 resolution.

B.10 VQA Inference

At inference time, the image, fixed at $t = 1$, and the tokenized question are provided as conditioning. Sampling is restricted to the answer span by allowing insertions only after the question tokens; the question itself remains fixed and is never modified.

We use the `FlowMatchEulerDiscreteScheduler` with shift $= 1.0$ for the text process and run 8 denoising steps, which we found sufficient for short answer spans. Decoding is greedy (top-$k = 1$,

temperature 0.7); classifier-free guidance is not applied (cfg $= 1.0$). The maximum answer sequence length is capped at 77 tokens. All results are obtained from the EMA checkpoint.

B.11 Inference Hyperparameters

All reported results use the EMA checkpoint. Sampling uses the `FlowMatchEulerDiscreteScheduler` with 28 denoising steps for both the image and text processes. For image-to-text captioning, the text scheduler uses shift $= 1.0$ with temperature 0.7 and top-$k = 1$ (greedy) decoding. For text-to-image generation, the image scheduler uses the default SD3 shift. Classifier-free guidance is not applied in the main comparisons (cfg $= 1.0$); qualitative examples in Section I use image-side CFG where noted.

C Evaluation Protocols

Unless noted, all metrics are computed on the EMA checkpoint with the inference settings of Section B.10 and the prompt suite of Section B.2. Image→text metrics are computed on a fixed 5k-pair held-out split of LAION-Aesthetic with single-reference Moondream captions; text→image metrics use the 1.9k-prompt suite.

Reference set for FID and CMMD. The reference set is the *base text-to-image model itself* sampled under identical conditions (same prompts, seed, scheduler, denoising steps, and CFG scale). This isolates the effect of multimodal uplift from generic backbone-quality differences: any deviation directly reflects retention or degradation of the pretrained image prior.

FID. Computed with the `clean-fid` package [42] using the standard Inception-V3 features, via `fid.compute_fid(base_fid_path, gen_dir, batch_size, num_workers=4)`.

CMMD. We compute CLIP Maximum Mean Discrepancy (CMMD) following Jayasumana et al. [23], using the public PyTorch implementation [2], which extracts image embeddings with `openai/clip-vit-large-patch14-336`.

CIDEr. For image-to-text evaluation on our held-out LAION-Aesthetic split, we compute CIDEr with the standard `pycocoevalcap` implementation [51], using the Moondream caption as the single reference for each image. For MS-COCO captioning, we use the standard validation annotations and score each generated caption against all five human reference captions for the corresponding image.

BERTScore. Computed via the `bert_score` package [60] with `lang="en"` (which selects `roberta-large` as the underlying scorer) and `rescale_with_baseline=True`; we report the F1 component, which can be negative under baseline rescaling.

CLIP score. Cosine similarity between CLIP image and text embeddings using `openai/clip-vit-base-patch32` [44], averaged over generated image-caption pairs.

VQA benchmarks. For VQAv2 [14], VizWiz [15], and OKVQA [39] we follow each benchmark's official evaluation protocol on its standard validation split.

$$\mathrm{acc}(a) = \min\left(\frac{\#\{\text{annotators agreeing with } a\}}{3},\ 1\right).$$

D Compute Budget

All experiments run on RTX A5000 (24 GB) GPUs in BF16 mixed precision; per-run training settings match Table 5. Table 6 lists the compute used for the experiments reported in this paper.

The 18 SD3-M runs cover the ablations in Section 5.2 (trainable parameters, conditioning pathways, gradient balancing, teacher matching, corruption schedule) and the final step- and time-matched checkpoints. Preliminary and failed configurations not reported in the paper are excluded from this total.

[2] https://github.com/sayakpaul/cmmd-pytorch

Table 6: Compute budget for the experiments reported in this paper, broken down by run group. Preliminary or failed configurations not included in the final results are excluded.

Run group	GPUs / run	Wall-clock / run	Total GPU-hours
SD3-M ablations + final (18 runs)	2	28 h	864
FLUX.1-dev (1 run)	3	36 h	108
DualDiff.-LoRA (1 run)	4	24 h	96
Total reported			**1,212**

E Motivating Analysis: Gradient Scale Mismatch

Our training objective combines an image regression loss and a text classification loss that share a transformer backbone. Because these losses induce gradients of different scale, naive weighting can cause one modality to dominate the shared-parameter updates. This appendix provides simple sanity-check calculations for the squared gradient norm with respect to the loss inputs, which motivates explicit loss reweighting such as the adaptive gradient balancing rule in Section 3.4.

Theorem E.1. *Let $\boldsymbol{X}, \boldsymbol{Y} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}_n)$ with $Cov(\boldsymbol{X}, \boldsymbol{Y}) = \sigma \boldsymbol{I}_n$, then the expected gradient norm is the following:*

$$\mathbb{E}_{\boldsymbol{X},\boldsymbol{Y}}\left[\|\nabla_{\boldsymbol{X}} \mathit{MSE}(\boldsymbol{X}, \boldsymbol{Y})\|_2^2\right] = \frac{8(1-\sigma)}{n} \tag{E.1}$$

Proof of Theorem E.1. Let $\boldsymbol{X}, \boldsymbol{Y}$ be defined as in Theorem E.1.

$$\begin{aligned}
\mathbb{E}_{\boldsymbol{X},\boldsymbol{Y}}\left[\|\nabla_{\boldsymbol{X}} \mathtt{MSE}(\boldsymbol{X}, \boldsymbol{Y})\|_2^2\right] &= \mathbb{E}_{\boldsymbol{X},\boldsymbol{Y}}\left[\left\|\frac{2}{n}(\boldsymbol{X}-\boldsymbol{Y})\right\|_2^2\right] \\
&= \frac{4}{n^2}\mathbb{E}_{\boldsymbol{X},\boldsymbol{Y}}\left[(\boldsymbol{X}-\boldsymbol{Y})^\top(\boldsymbol{X}-\boldsymbol{Y})\right] \\
&= \frac{4}{n^2}\left(\mathbb{E}_{\boldsymbol{X},\boldsymbol{Y}}\left[\boldsymbol{X}^\top\boldsymbol{X}\right] + \mathbb{E}_{\boldsymbol{X},\boldsymbol{Y}}\left[\boldsymbol{Y}^\top\boldsymbol{Y}\right] - 2\mathbb{E}_{\boldsymbol{X},\boldsymbol{Y}}\left[\boldsymbol{X}^\top\boldsymbol{Y}\right]\right) \\
&= \frac{4}{n^2}\left(n + n - 2n\right) \\
&= \frac{8(1-\sigma)}{n}
\end{aligned}$$

□

Theorem E.2. *Let $\boldsymbol{Z} \in \mathbb{R}^n$ be a random logits vector, $\boldsymbol{P} = \mathit{softmax}(\boldsymbol{Z}) \in \triangle^{n-1}$, and let $\boldsymbol{Q} = e_Y$ be a one-hot encoding of the target label $Y \in [n]$. Further, let $\boldsymbol{P}$ and $\boldsymbol{Q}$ agree in expectation by $\mathbb{E}\left[\boldsymbol{P}^\top\boldsymbol{Q}\right] = \sigma$. Then*

$$\frac{n}{n-1}(1-\sigma)^2 \;\le\; \mathbb{E}\left[\|\nabla_{\boldsymbol{Z}} \mathit{CE}(\boldsymbol{P}, \boldsymbol{Q})\|_2^2\right] \;\le\; 2(1-\sigma). \tag{E.2}$$

Proof of Theorem E.2. Let $\boldsymbol{P}, \boldsymbol{Q}$ be defined as in Theorem E.2. Since $\nabla_{\boldsymbol{Z}} \mathtt{CE}(\boldsymbol{P}, \boldsymbol{Q}) = \boldsymbol{P} - \boldsymbol{Q}$, we have

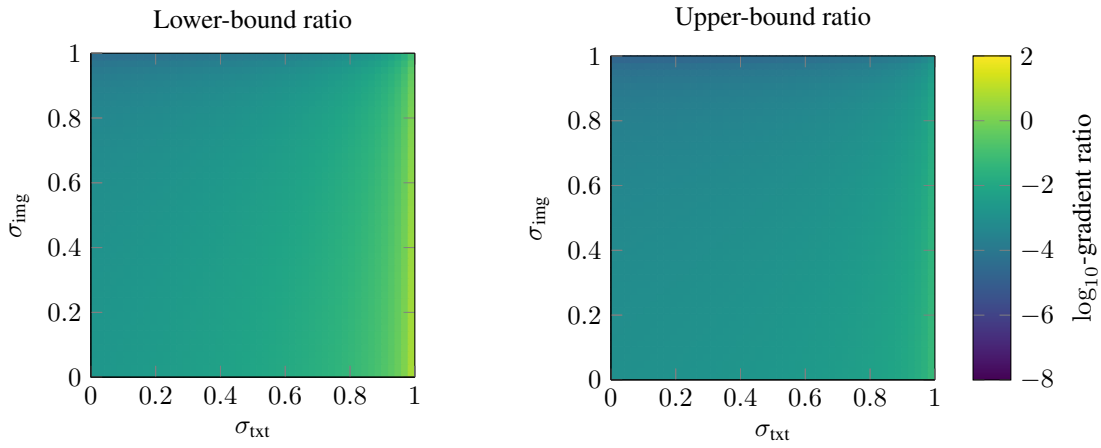
$$\|\boldsymbol{P} - \boldsymbol{Q}\|_2^2 = \|\boldsymbol{P} - e_Y\|_2^2 = (1 - P_Y)^2 + \sum_{j \neq Y} P_j^2.$$

For the upper bound, note that $\sum_{j\neq Y} P_j^2 \le \sum_{j \neq Y} P_j$ and $(1-P_Y)^2 \le 1 - P_Y$ since $P \in \triangle^{n-1}$, hence

$$\|\boldsymbol{P} - e_Y\|_2^2 \le \sum_{j \neq Y} P_j + (1 - P_Y) = 2(1 - P_Y)$$

Taking expectation yields

$$\mathbb{E}\left[\|\boldsymbol{P} - \boldsymbol{Q}\|_2^2\right] \le 2\left(1 - \mathbb{E}[P_Y]\right) = 2(1-\sigma).$$



$$\log_{10}\left(\frac{\|\nabla_{\theta_{\text{sh}}}\mathcal{L}_{\text{img}}\|_2}{\|\nabla_{\theta_{\text{sh}}}\mathcal{L}_{\text{txt}}\|_2+\epsilon}\right)$$

Figure 9: Log-ratio between the expected MSE gradient norm (image branch, Theorem E.1) and the lower/upper bounds on the cross-entropy gradient norm (text branch, Theorem E.2), as a function of the agreement levels σ_{img} and σ_{txt} for latent dimension $N = 4096$. Negative regions, which dominate early training when text predictions are poor and the image prior is already accurate, indicate that the text-branch gradient outweighs the image-branch gradient and motivates the loss reweighting in Section 3.4.

For the lower bound, by Cauchy–Schwarz on the $n-1$ coordinates $j \in [n]$ with $j \neq Y$,

$$\sum_{j\neq Y} P_j^2 \geq \frac{\left(\sum_{j\neq Y} P_j\right)^2}{n-1} = \frac{(1-P_Y)^2}{n-1}.$$

Therefore,

$$\|\boldsymbol{P} - e_Y\|_2^2 \geq (1-P_Y)^2 + \frac{(1-P_Y)^2}{n-1} = \frac{n}{n-1}(1-P_Y)^2.$$

Taking expectation and using Jensen's inequality,

$$\mathbb{E}\left[\|\boldsymbol{P}-\boldsymbol{Q}\|_2^2\right] \geq \frac{n}{n-1}\mathbb{E}\left[(1-P_Y)^2\right] \geq \frac{n}{n-1}\left(1-\mathbb{E}[P_Y]\right)^2 = \frac{n}{n-1}(1-\sigma)^2.$$

Combining both bounds completes the proof. □

Theorems E.1 and E.2 highlight a fundamental scale mismatch between the two loss terms. For the image regression objective, the squared gradient w.r.t. the prediction scales as $\mathbb{E}\left[\|\nabla\text{MSE}\|_2^2\right] = \frac{8(1-\sigma_{\text{img}})}{n}$ and therefore decays as $O(1/n)$. In our setting, SD3 is trained on 256×256 images and operates in a VAE latent space of size $n = 4 \times 32 \times 32 = 4096$, making this effect non-negligible.

In contrast, for token cross-entropy with one-hot targets, the squared gradient w.r.t. logits is bounded by $\frac{n}{n-1}(1-\sigma_{\text{txt}})^2 \leq \mathbb{E}\left[\|\nabla\text{CE}\|_2^2\right] \leq 2(1-\sigma_{\text{txt}})$, which is $O(1)$ and essentially independent of the vocabulary size for large n. Early in training, text prediction is poor, so $\sigma_{\text{txt}} \approx 0$, while the pretrained image branch is already accurate, yielding $\sigma_{\text{img}} \approx 1$; consequently, the text objective can induce substantially larger gradients than the image objective on shared parameters (after applying the chain rule). Figure 9 visualizes this disparity by plotting the $\log_{10}$-ratio between the expected image gradient norm and the lower/upper bounds for the text gradient norm.

While the assumptions of Theorems E.1 and E.2 are idealized, they provide a useful explanation for why explicit loss balancing is required in practice. Consistent with this analysis, our adaptive weighting in Section 3.4 stabilizes to $\lambda_{\text{txt}} \approx 0.04$ over training, indicating a persistent order-of-magnitude correction between the two objectives.

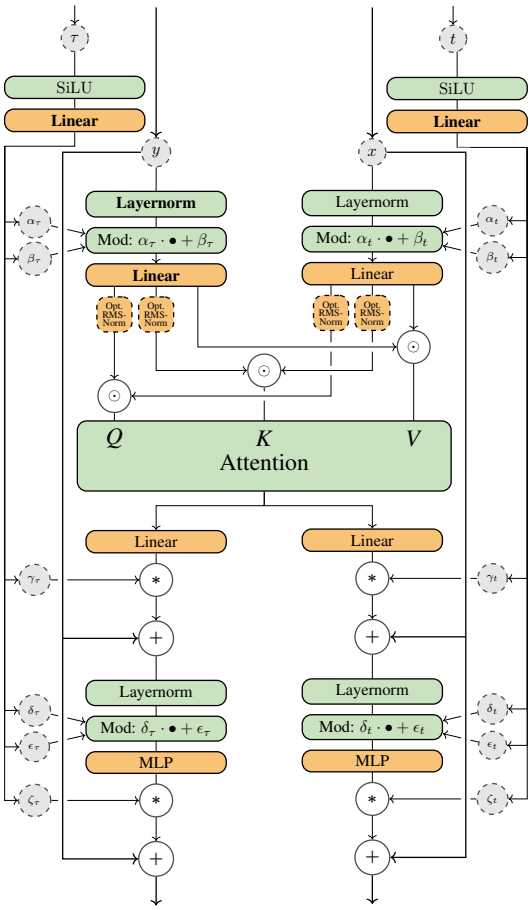


Figure 10: MM-DiT block adapted from Stable Diffusion 3 for joint image–text generation. We add text-time conditioning τ alongside the original image-time conditioning t, with both timesteps embedded separately and injected through the existing AdaLN-style modulation interface. Yellow blocks denote LoRA-augmented linear layers; green blocks denote frozen auxiliary operations.

F Architecture

Stable Diffusion 3. Our SD3 adaptation follows Section 3.2 and Figure 3. The original MM-DiT blocks are conditioned on the image timestep through AdaLN-style modulation. We add a second timestep input for the text corruption level τ: image time t and text time τ are embedded separately, passed through MLPs, and injected through the same modulation interface. Apart from this dual-timestep conditioning and the LoRA-augmented linear layers shown in Figure 10, the SD3 transformer architecture is unchanged.

FLUX.1 [dev]. The FLUX.1 [dev] adaptation uses the same dual-timestep principle. FLUX processes image tokens and T5 text-token embeddings in the main transformer, while also using pooled CLIP text embeddings for global conditioning. We therefore add separate processors for t and τ and inject both through the model's existing modulation pathway. As in SD3, the backbone is frozen except for LoRA-augmented linear layers, while the text-generation heads and modality-specific timestep processors are newly trained; see Figure 11.

G Multi-encoder Conditioning

Modern text-to-image backbones often combine token-level text features with pooled or global text embeddings for conditioning. In SD3, these signals come from multiple frozen encoders, including T5-XXL and CLIP; in FLUX.1, the main text stream uses T5 embeddings while pooled CLIP embeddings provide additional global conditioning. During multimodal uplift, we therefore need to

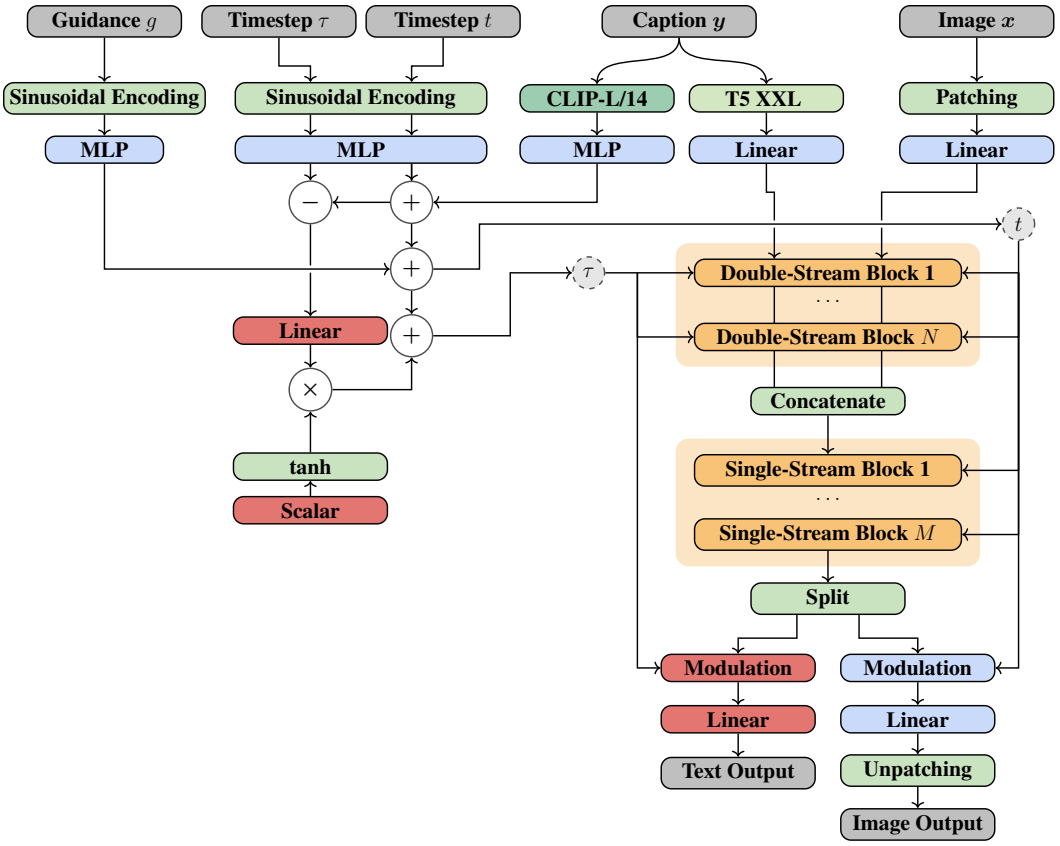


Figure 11: Multimodal DiT architecture adapted from FLUX.1 [dev]. Noisy text tokens and image patches are processed jointly together with their modality-specific timesteps (τ, t) to predict cross-modal flows. Blue: frozen pretrained components; yellow: LoRA-augmented linear layers; red: newly added trainable modules (text head and timestep processors); green: auxiliary operations.

provide compatible inputs to each conditioning pathway while defining the discrete text process in a single tokenizer space.

Design choice. We perform discrete text diffusion in the T5 tokenizer space. This keeps the generative state aligned with a text encoder that is trained for token-level modeling, while still allowing us to reuse the pretrained CLIP encoders as additional conditioning signals.

Key idea: decode $\tilde{\boldsymbol{y}}_\tau$ to a natural-language string and re-tokenize. At training (and sampling) time we have a partially observed/deleted T5 token sequence $\tilde{\boldsymbol{y}}_\tau \sim q_\tau(\cdot \mid \boldsymbol{y})$. Because our corruption is *deletion-to-empty* (not mask-token corruption), $\tilde{\boldsymbol{y}}_\tau$ is a subsequence of the original prompt and typically detokenizes into a reasonably natural fragment. We exploit this to obtain inputs for auxiliary encoders:

1. **T5 branch (diffusion alphabet):** feed $\tilde{\boldsymbol{y}}_\tau$ directly to the frozen T5 encoder to obtain token embeddings for the shared transformer.
2. **CLIP branches (auxiliary conditioning):** decode $\tilde{\boldsymbol{y}}_\tau$ to a string $\tilde{s}_\tau = \mathrm{decode}_{\mathrm{T5}}(\tilde{\boldsymbol{y}}_\tau)$ (dropping special tokens), then re-tokenize $\tilde{s}_\tau$ with each CLIP tokenizer and feed the resulting IDs to the corresponding frozen CLIP encoders (including pooled embeddings).

This yields a simple, deterministic, and cheap mapping between encoder stacks while keeping all text encoders frozen.

Why deletion helps. Mask-based discrete diffusion injects artificial symbols (e.g., `<mask>`) and positional patterns that the pretrained CLIP encoders were never trained on, often requiring explicit mask-token retraining. In contrast, deletion produces strings composed of *valid* natural-language tokens; even when the fragment is ungrammatical, CLIP-style encoders are typically robust enough for conditioning.

Limitations. The decoded fragment $\tilde{s}_\tau$ can be incomplete or slightly ungrammatical, especially at small τ. In our setup this is acceptable because (i) these auxiliary encoders provide *conditioning* rather than a supervised target, and (ii) the T5 branch remains the primary text state that the model learns to denoise. Table 7 shows typical decoded fragments produced by deletion at different τ. These decoded strings are the inputs re-tokenized for auxiliary encoders.

H Results

This appendix section presents additional ablation studies and analysis beyond the main paper results. We examine the impact of different architectural and training choices, including the effect of classifier-free guidance (CFG), teacher-matching strategies, and the corruption schedule used during training. These results complement the main paper evaluation and provide deeper insights into the design decisions that contribute to FullFlow's strong performance.

H.1 Ablation 1: Classifier-Free Guidance

Training with the π_{ind} schedule (Equation (3.3)) exposes the model to jointly corrupted states across the full two-timestep space. In particular, the model sees regimes where one modality carries little or no information while the other remains informative. This provides unconditional or near-unconditional predictors for both the image and text branches without adding a separate conditioning-dropout objective. At inference time, these predictors can be combined with their conditional counterparts to implement classifier-free guidance (CFG) for either text-to-image or image-to-text generation.

Dual Diffusion [31] instead introduces an explicit conditioning-dropout hyperparameter, which controls the probability of fully deleting the conditioning modality during training in order to preserve an unconditional prediction branch. In our formulation, the same functionality arises naturally from the two-timestep training space induced by π_{ind}.

Text-to-image. For text-to-image generation we condition on a clean caption $\boldsymbol{y}$ and evolve the image from Gaussian noise $\boldsymbol{x}_0 \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ along $t : 0 \to 1$. We define

$$\boldsymbol{y}_1 \coloneqq \boldsymbol{y}, \qquad \boldsymbol{y}_0 \coloneqq \varepsilon,$$

corresponding to fully informative vs. fully uninformative text contexts. The model provides

$$v_\theta^{\text{cond}} \coloneqq v_\theta(\boldsymbol{x}_t, t, \boldsymbol{y}_1, \tau{=}1),$$
$$v_\theta^{\text{uncond}} \coloneqq v_\theta(\boldsymbol{x}_t, t, \boldsymbol{y}_0, \tau{=}0),$$

and we use the standard CFG combination

$$v_{\text{cfg}} = v_\theta^{\text{uncond}} + \gamma \cdot \left(v_\theta^{\text{cond}} - v_\theta^{\text{uncond}}\right),$$

with guidance scale $\gamma \geq 1$. This directly steers the image flow toward captions that better match $\boldsymbol{y}$ without extra networks or retraining.

Image-to-text. The same construction applies symmetrically to image-to-text generation, but now the discrete nature of Edit Flows requires combining the parameters of the reverse-time CTMC rather than a Euclidean velocity. We condition on a clean image $\boldsymbol{x}_1 \coloneqq \boldsymbol{x}$ and evolve the text from the empty sequence ε along $\tau : 0 \to 1$, using

$$\boldsymbol{x}_1 \coloneqq \boldsymbol{x}, \qquad \boldsymbol{x}_0 \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}),$$

as fully informative vs. fully uninformative image contexts. At each retained position i, the backbone produces a conditional and an unconditional pair of insertion rate and token distribution:

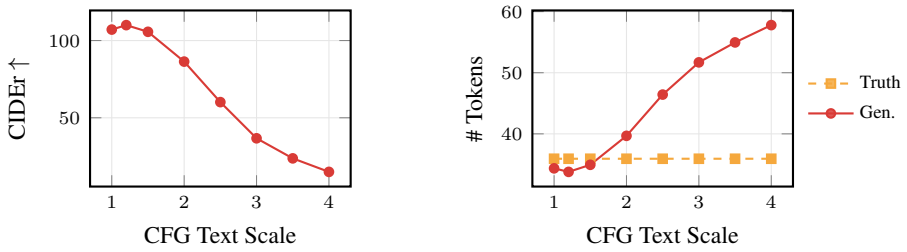
$$\lambda_\theta^{i,\text{cond}} \coloneqq \lambda_\theta^i\left(\tilde{\boldsymbol{y}}_\tau, \tau, \boldsymbol{x}_1, t{=}1\right), \quad w_\theta^{i,\text{cond}}(\cdot) \coloneqq w_\theta^i\left(\cdot \mid \tilde{\boldsymbol{y}}_\tau, \tau, \boldsymbol{x}_1, t{=}1\right),$$
$$\lambda_\theta^{i,\text{uncond}} \coloneqq \lambda_\theta^i\left(\tilde{\boldsymbol{y}}_\tau, \tau, \boldsymbol{x}_0, t{=}0\right) \quad w_\theta^{i,\text{uncond}}(\cdot) \coloneqq w_\theta^i\left(\cdot \mid \tilde{\boldsymbol{y}}_\tau, \tau, \boldsymbol{x}_0, t{=}0\right).$$

Since insertion rates are nonnegative and token distributions are categorical, we combine them multiplicatively in log-space, which is the discrete analogue of the additive velocity combination above:

$$\log \lambda_\theta^{i,\text{cfg}} = \log \lambda_\theta^{i,\text{uncond}} + \gamma \cdot \left(\log \lambda_\theta^{i,\text{cond}} - \log \lambda_\theta^{i,\text{uncond}}\right),$$

$$w_\theta^{i,\mathrm{cfg}}(v) \propto w_\theta^{i,\mathrm{uncond}}(v)^{1-\gamma} \cdot w_\theta^{i,\mathrm{cond}}(v)^{\gamma},$$

for each candidate token $v \in V$, with $w_\theta^{i,\mathrm{cfg}}$ renormalized over the vocabulary. The guided rate $\lambda_\theta^{i,\mathrm{cfg}}$ and distribution $w_\theta^{i,\mathrm{cfg}}$ are then plugged into the same insertion sampler used at vanilla inference, steering the discrete text trajectory toward sequences that are more strongly explained by the conditioning image $\boldsymbol{x}$ at no additional training cost.



(a) **CIDEr Score ↑** (b) **Caption Length**

Figure 12: Effect of image-to-text classifier-free guidance scale γ on caption quality and length. Mild guidance ($\gamma \approx 1.2$) improves CIDEr and brings caption length closer to the reference; stronger guidance over-amplifies high-probability insertions in the reverse CTMC, producing overly verbose captions and degrading quality.

We evaluate the effect of image-to-text CFG on caption generation in Figure 12. Unless stated otherwise, all main results in this paper use unguided decoding, i.e. $\gamma = 1$, for simplicity and to avoid introducing an additional inference-time hyperparameter. Nevertheless, CFG provides a useful post-hoc control mechanism. A mild guidance scale, $\gamma = 1.2$, improves CIDEr from 107.1 to 110.0 while slightly shortening the generated captions from 34.4 to 33.8 tokens on average, bringing them closer to the reference length of 35.9 tokens. However, stronger guidance quickly becomes detrimental: as γ increases beyond 1.5, the model generates increasingly long captions, reaching 57.8 tokens at $\gamma = 4.0$, while CIDEr drops monotonically to 15.0. This suggests that moderate CFG can improve caption-image alignment, but excessive guidance over-amplifies high-probability insertions in the reverse CTMC, producing overly verbose captions and degrading caption quality.

H.2 Ablation 2: Teacher-matching

Qualitative examples of the effect of different teacher-matching styles vs the standard rectified flow formulation. As confirmed by the metrics presented in the main part, the standard rectified flow formulation yields high distortion which is likely the result of a training distribution mismatch between our training data and the data the model was pretrained on. Both teacher-matching versions are able to provide coherent images. However, clean teacher-matching introduces a lot of artifacts at first and seems overall more unstable, while same-noise teacher-matching stays very stable, and is hence the standard choice.

H.3 Ablation 3: Corruption Schedule Ablation: Extended Metrics

Figure 14 extends the corruption-schedule ablation from Section 5.2 to four additional metrics: CLIP score (image–text alignment), T5-token union (lexical overlap between the generated and reference captions), BERTScore-F1 (semantic similarity), and GPT-2 perplexity (caption fluency). A detailed quantitative breakdown of these metrics is provided in Table 8. The qualitative pattern from the main-paper CIDEr curves (Figure 5) replicates consistently across all four metrics. Pure π_{ac} (orange) converges fastest early in training because it supervises captioning directly by always conditioning on clean images; this advantage is most visible in GPT-2 perplexity, where π_{ac} achieves lower perplexity from the very first checkpoint. Pure π_{ind} (blue) learns more slowly on captioning-specific metrics but builds richer cross-modal representations by training on all $(t, \tau) \in [0, 1]^2$ jointly. After switching from π_{ind} to π_{ac} (red), performance improves sharply and reaches the final pure-π_{ac} level in approximately half the remaining training budget, regardless of the exact switch point between 75k and 175k steps. These results confirm that the mixed-to-alternating schedule is the preferred default across all evaluated metrics.

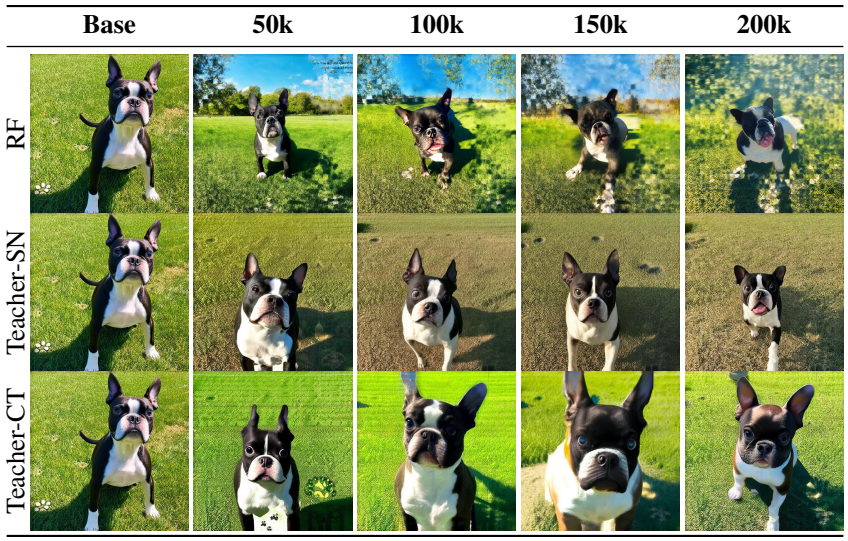


Figure 13: Qualitative effect of the image-supervision target on text-to-image generation across training checkpoints (50k–200k steps). RF: standard rectified flow; Teacher-CT: clean-text teacher matching; Teacher-SN: same-noise teacher matching. RF accumulates distortion, Teacher-CT introduces transient artifacts and is unstable, while Teacher-SN preserves the visual prior throughout training and is therefore our default.

I Qualitative Examples

This appendix collects qualitative samples from the SD3 and FLUX.1-dev FullFlow models on unseen evaluation inputs, complementing the quantitative results in Section 5. Figures 15 to 17 show image-to-text captions, Figures 18 and 19 compare text-to-image samples between each base model and its FullFlow finetune, Figure 20 shows VQAv2 predictions, and Figures 21 to 26 illustrate joint image–text generation along the $\tau = t^{2^p}$ trajectories from Section 5.6. All samples are generated with the inference settings of Section B.10 on inputs that were not seen during training.

I.1 Joint Generation: Qualitative Examples

Figures 21 to 26 show additional examples of FullFlow's joint generation mode, where image and text are sampled simultaneously without conditioning on either modality. We sweep the trajectory parameter $p \in \{-5, -2.5, 0, 2.5, 5\}$ using $\tau = t^{2^p}$ as detailed in Section 5.6. Negative values of p denoise text earlier than the image, positive values denoise the image earlier than text, and $p = 0$ follows the diagonal trajectory where both noise levels advance together.

The qualitative examples are consistent with the CLIP-score trend in Figure 6: the balanced trajectory ($p = 0$) yields the most coherent image–text pairs, with captions describing the main objects, colors, and spatial relations in the image. Text-first trajectories ($p < 0$) remain competitive but can commit early to generic semantic content that the image then follows. Image-first trajectories ($p > 0$) often defer language generation until the visual scene is mostly formed; at large positive values, captions tend to emphasize local visual details rather than the overall scene. Figure 21 shows that generated captions also become longer for larger p, suggesting that delayed text denoising encourages more verbose descriptions.

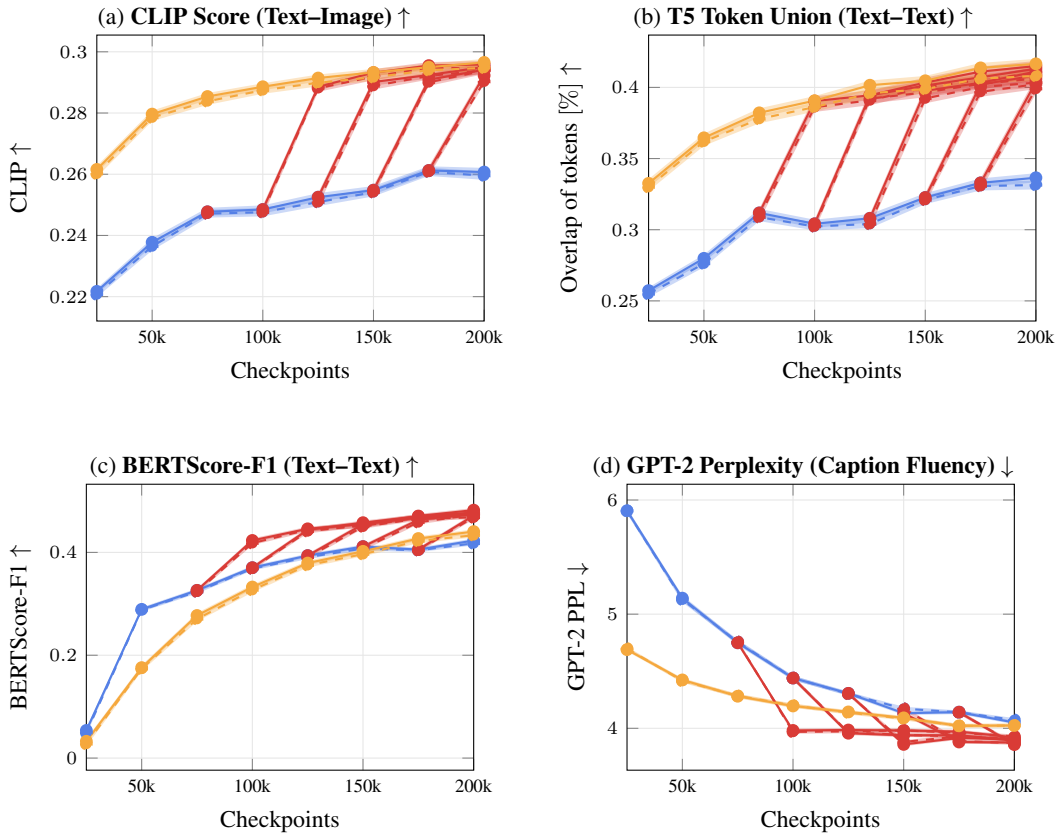


Figure 14: Extended corruption-schedule ablation from Section 5.2, showing four additional captioning metrics beyond CIDEr. **yellow**: pure alternating-clean (π_{ac}); **blue**: pure mixed-corruption (π_{ind}); **red**: five independent runs that switch from π_{ind} to π_{ac} at 75k, 100k, 125k, 150k, and 175k steps. Solid lines: train; dashed lines: held-out validation. Shaded bands: 95% CI over 5k samples. Across all four metrics, the mixed-to-alternating schedule matches or exceeds pure π_{ac} at convergence while requiring only half the training budget after the switch.

Table 7: Multi-encoder bridging via deletion at four retention levels τ. The original prompt s is corrupted in T5-tokenizer space to $\tilde{\boldsymbol{y}}_\tau$, decoded back into a string $\tilde{s}_\tau$, and re-tokenized for auxiliary encoders such as CLIP. Lower τ retains fewer tokens, but the surviving fragments remain natural-language tokens rather than mask symbols.

τ	**Original prompt** s	**Decoded fragment** $\tilde{s}_\tau$
0.8	A vintage green automobile with a distinctive grille and round headlights is parked on a paved surface, surrounded by greenery.	A vintage automobile witha distinctivee and round-light isparked on a, surrounded by greenery.
0.8	A close-up image of a layered cake with whipped cream and fresh raspberries on top, with a slice being cut out and a spatula resting on the side.	A close-up image of a layered cake withwhipped cream and raspberries on top with a slice cut out and a spatul resting on the side.
0.8	The image features a repeating pattern of green holly leaves and red berries on a light green background, with some scattered dots.	The image features a repeat pattern of green holly leaves and red berries on light green background, with some scattered dots.
0.8	Four well-dressed men in suits are posing for a photo, with one holding a glass of champagne.	Four well-dresse men in suits are posing for photo, one holding a glass of champagne.
0.6	The image features a circular rug with a vibrant array of fruits, including bananas, apples, oranges, strawberries, grapes, pineapple, and watermelon, all depicted in a colorful and playful manner.	The image features a circular a vibrant array of fruits includings, apples, strawberries,, pineap-plemel, all depicteda colorful and manner
0.6	A red enamel coffee pot with a white interior sits on a wooden surface, its handle pointing towards the right.	A enamel pot with a whites on wooden, its the right.
0.6	A watercolor illustration depicts a jar of jam with a purple ribbon tied around its neck, set against a white background with splashes of red and pink.	watercolor of jam with a purple tied its neck, set against a white splashes of red and pink.
0.6	The image depicts a whimsical beach scene with a vintage camper, a dog, and beach paraphernalia, all set against a backdrop of a cloudy sky.	images beach scene with vintage camper,a dog beach paran, set against ay
0.4	A man is holding a blue and white device, possibly a blood glucose meter, and appears to be checking his finger for a drop of blood.	A white, possiblyameter finger drop
0.4	The image shows a vintage wooden cabinet with glass doors, featuring ornate carvings and brass hardware.	The shows cabinet with, featuringnate and brass hardware.
0.4	The image features a vintage side table with a wooden top and a shelf, set against a plain white background.	image table with wooden topa shelf, against
0.4	The image shows a wooden platter filled with a variety of fresh fruits and vegetables, including strawberries, asparagus, lemon wedges, avocado, and grilled chicken.	The image with variety wedge avocado, and grilled
0.2	Three cupcakes with colorful frosting are displayed on a white surface, with the central cupcake having a swirl of chocolate and vanilla frosting.	central a of and.
0.2	The image features a slice of cake with intricate frosting on a plate, with a larger cake in the background on a stand.	with in
0.2	The image shows the intricate inner workings of a vintage gold watch, featuring ornate engravings and a central blue gemstone.	engraving
0.2	The image shows a colorful, intricately embroidered garment with a vibrant pattern, displayed on a mannequin.	showsa colorful onamann

Table 8: Final-checkpoint (200k steps) statistics for the corruption-schedule ablation of Figure 14, complementing the curves with exact numerical values. For each schedule, split, and captioning metric we report the mean over n=5000 validation samples, the sample standard deviation, and the half-width of the 95% confidence interval used as the shaded band in Figure 14.

		CIDEr ↑			**CLIP ↑**			**T5 Tok. Union ↑**			**BERTScore-F1 ↑**			**GPT-PPL ↓**		
Schedule	Split	mean	std	95% CI	mean	std	95% CI	mean	std	95% CI	mean	std	95% CI	mean	std	95% CI
π_{ind}	train	66.78	77.74	2.25	0.26	0.05	0.00	0.34	0.11	0.00	0.42	0.14	0.00	4.05	0.55	0.02
	val	65.43	76.87	2.21	0.26	0.05	0.00	0.33	0.11	0.00	0.42	0.14	0.00	4.07	0.55	0.02
switch @ 75k	train	114.70	104.30	3.02	–	–	–	0.41	0.12	0.00	0.48	0.14	0.00	3.92	0.56	0.02
	val	111.80	103.19	2.97	–	–	–	0.41	0.12	0.00	0.48	0.14	0.00	3.93	0.56	0.02
switch @ 100k	train	111.75	101.57	2.94	0.30	0.04	0.00	0.41	0.12	0.00	0.48	0.14	0.00	3.89	0.55	0.02
	val	107.63	100.21	2.88	0.29	0.04	0.00	0.40	0.12	0.00	0.47	0.14	0.00	3.93	0.54	0.02
switch @ 125k	train	110.71	101.19	2.93	0.30	0.04	0.00	0.41	0.12	0.00	0.47	0.14	0.00	3.90	0.54	0.02
	val	108.76	100.40	2.89	0.29	0.04	0.00	0.40	0.12	0.00	0.47	0.14	0.00	3.91	0.55	0.02
switch @ 150k	train	84.94	89.82	2.60	0.29	0.04	0.00	0.40	0.12	0.00	0.43	0.14	0.00	3.83	0.53	0.02
	val	84.68	91.90	2.64	0.29	0.04	0.00	0.40	0.12	0.00	0.43	0.14	0.00	3.83	0.54	0.02
switch @ 175k	train	108.61	99.52	2.88	0.29	0.04	0.00	0.40	0.12	0.00	0.47	0.14	0.00	3.86	0.54	0.02
	val	107.11	100.64	2.89	0.29	0.04	0.00	0.40	0.12	0.00	0.47	0.14	0.00	3.86	0.54	0.02
π_{ac}	train 99.38	94.95	2.75	0.30	0.04	0.00	0.42	0.12	0.00	0.44	0.15	0.00	4.02	0.59	0.02	
	val 96.97	95.88	2.76	0.29	0.04	0.00	0.41	0.12	0.00	0.43	0.15	0.00	4.02	0.58	0.02	

Table 9: Image-to-text generation statistics on SD3 with matched data, LoRA rank, and training setup, comparing the DualDiff-LoRA baseline against FullFlow at matched training steps and matched wall-clock time. All metrics are computed against held-out Moondream captions on the validation split.

	CIDEr ↑			**CLIP ↑**			**T5 Tok. Union ↑**			**BERTScore-F1 ↑**			**GPT-PPL ↓**		
Model	mean	std	95% CI	mean	std	95% CI	mean	std	95% CI	mean	std	95% CI	mean	std	95% CI
DualDiff.-LoRA	2.06	5.36	0.15	–	–	–	0.11	0.06	0.00	–0.28	0.78	0.02	7.66	2.30	0.06
FullFlow, step-matched	13.16	25.73	0.74	0.26	0.04	0.00	0.33	0.10	0.00	0.03	0.17	0.00	4.70	0.56	0.02
FullFlow, time-matched	107.11	100.64	2.89	0.29	0.04	0.00	0.40	0.12	0.00	0.47	0.14	0.00	3.86	0.54	0.02

Table 10: Visual question answering benchmarks. Top group: dedicated autoregressive VLMs; middle group: unified models that handle generation and understanding jointly; bottom group: flow-matching unified models. With only 130M trainable parameters and lightweight downstream adaptation, FullFlow remains competitive with much larger unified systems while substantially outperforming the closest flow-matching baseline (DualDiff) on VizWiz.

Model	**Params** # trainable	**Text** Backbone	**Image** Backbone	**MS-COCO** CIDEr ↑	**VQAv2** Acc. ↑	**VizWiz** Acc. ↑	**OKVQA** Acc. ↑
InternVL-2.0 [10]	8B	AR	-	-	-	62.9	62.9
LLaVA-Next [27]	13B	AR	-	-	82.8	60.5	-
BLIP [28]	13B	AR	-	-	65.0	19.6	-
IDEFICS [26]	9B	AR	-	-	50.9	-	-
QWEN-VL [4]	7B	AR	-	-	78.2	38.9	-
OpenFlamingo [3]	9B	AR	-	65.5	43.5	-	-
Flamingo [2]	9B	AR	-	79.4	51.8	28.8	44.7
CM3Leon [58]	7B	AR	AR	61.6	47.6	37.6	23.8
Chameleon [50]	7B	AR	AR	18.0	-	-	-
LWM [33]	7B	AR	AR	-	55.8	11.6	-
Show-O (256×256) [54]	1.3B	AR	FM	-	64.7	-	-
Show-O (512×512) [54]	1.3B	AR	FM	-	69.4	-	-
Transfusion [61]	7B	AR	FM	29.0	-	-	-
DualDiff (256×256) [31]	2B	FM	FM	-	59.5	19.4	28.5
DualDiff (512×512) [31]	2B	FM	FM	56.2	60.1	29.9	25.3
FullFlow (Ours)	130M	FM	FM	54.93	56.5	50.7	23.3

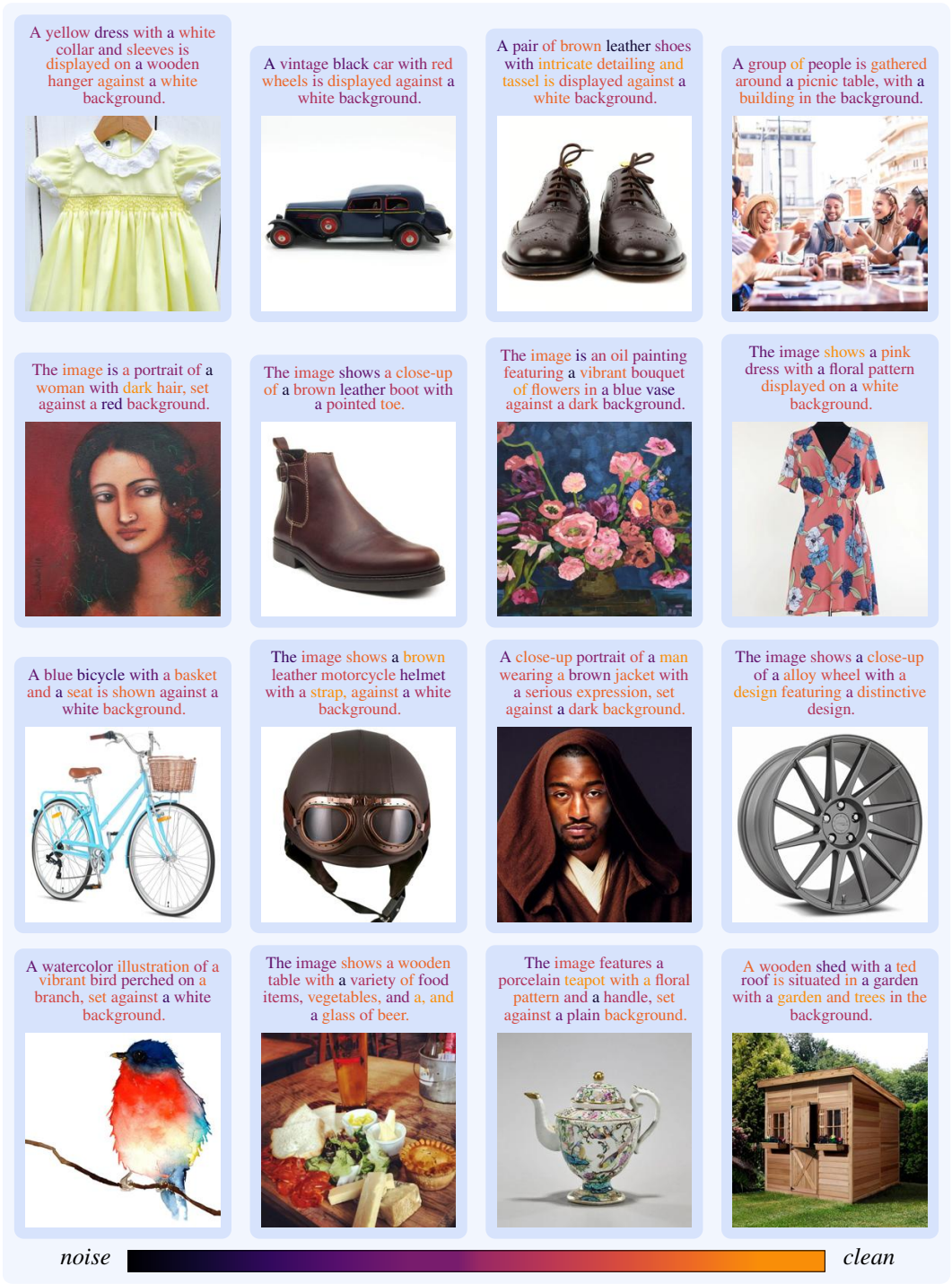


Figure 15: Image-to-text captions from the SD3 FullFlow model on unseen LAION-Aesthetic images (part 1/2). Each tile shows the input image and the greedy caption sampled by the model.

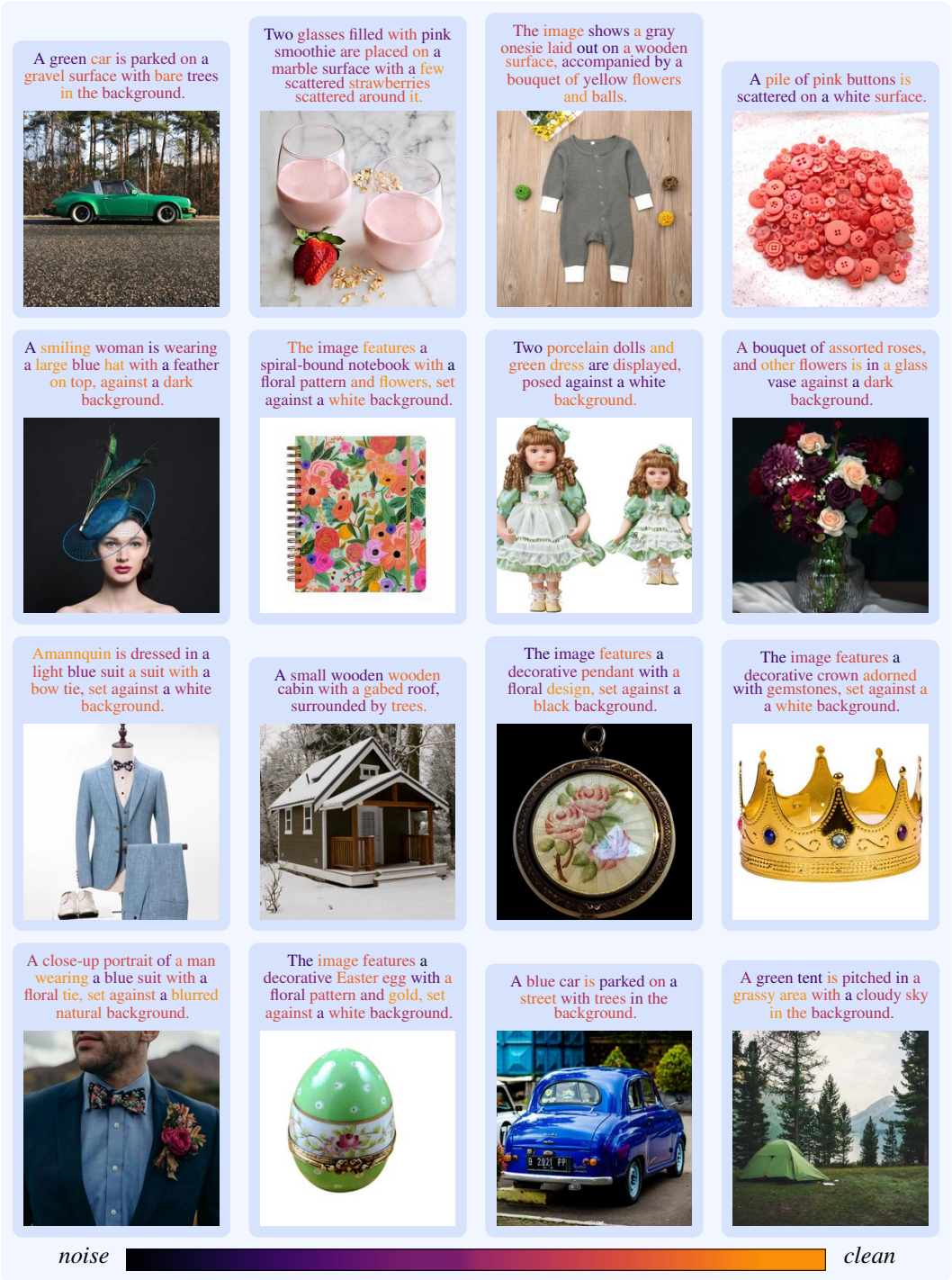


Figure 16: Image-to-text captions from the SD3 FullFlow model on unseen LAION-Aesthetic images (part 2/2). Continuation of Figure 15.

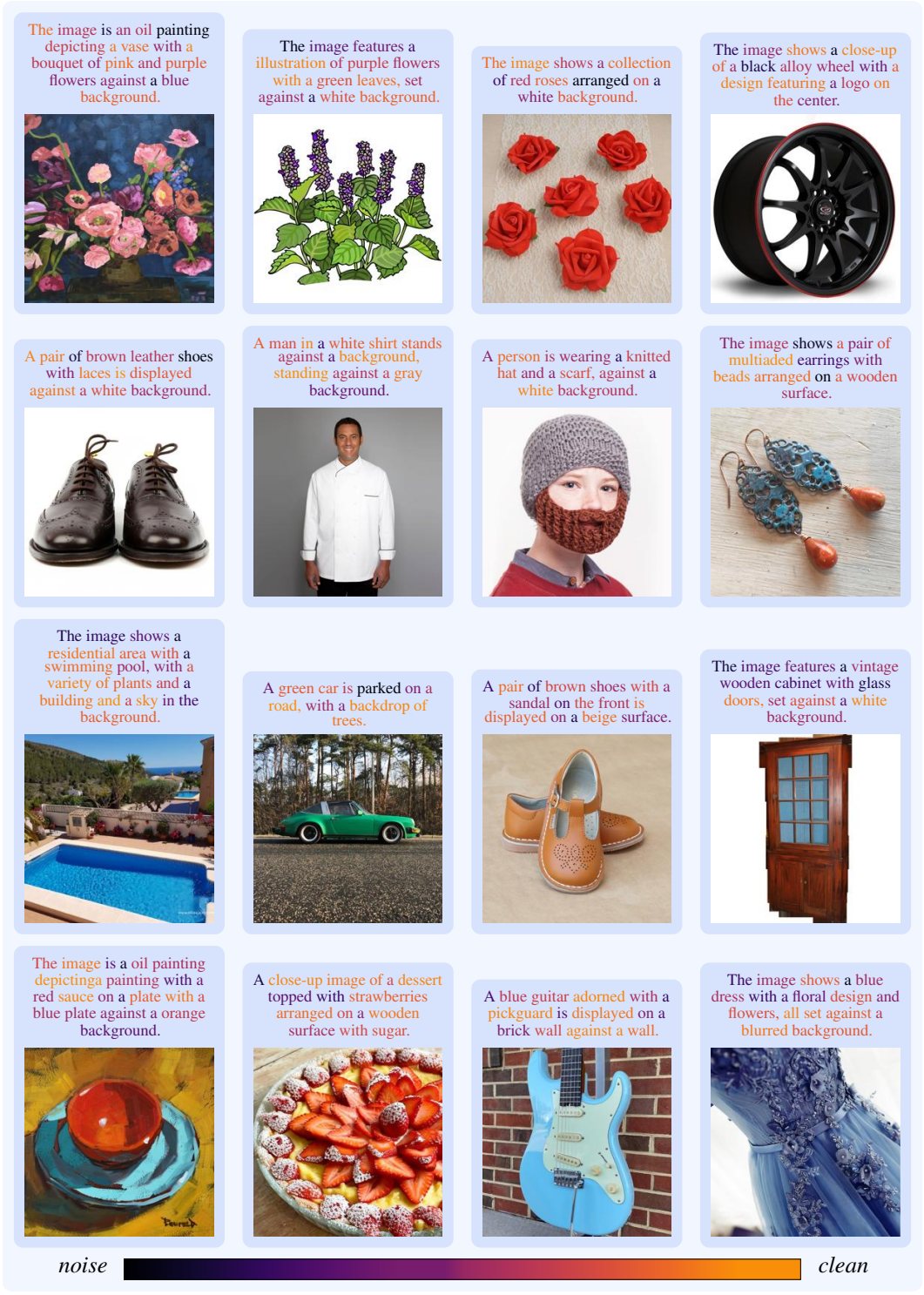


Figure 17: Image-to-text captions from the FLUX.1 [dev] FullFlow model on unseen LAION-Aesthetic images, demonstrating that the same recipe transfers to a different rectified-flow backbone.

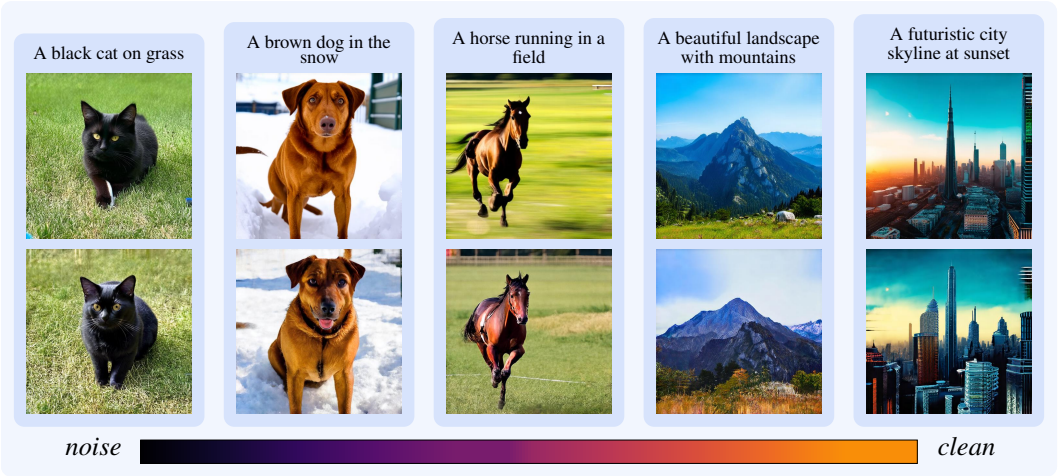


Figure 18: Text-to-image samples from the SD3 base model (top row) and our FullFlow SD3 finetune (bottom row) under identical prompts, seeds, and schedulers. Differences between rows isolate the effect of the multimodal uplift on the visual prior.

Figure 19: Text-to-image samples from the FLUX.1 [dev] base model (top row) and our FullFlow FLUX.1 finetune (bottom row) under identical prompts, seeds, and schedulers, mirroring the SD3 comparison in Figure 18.

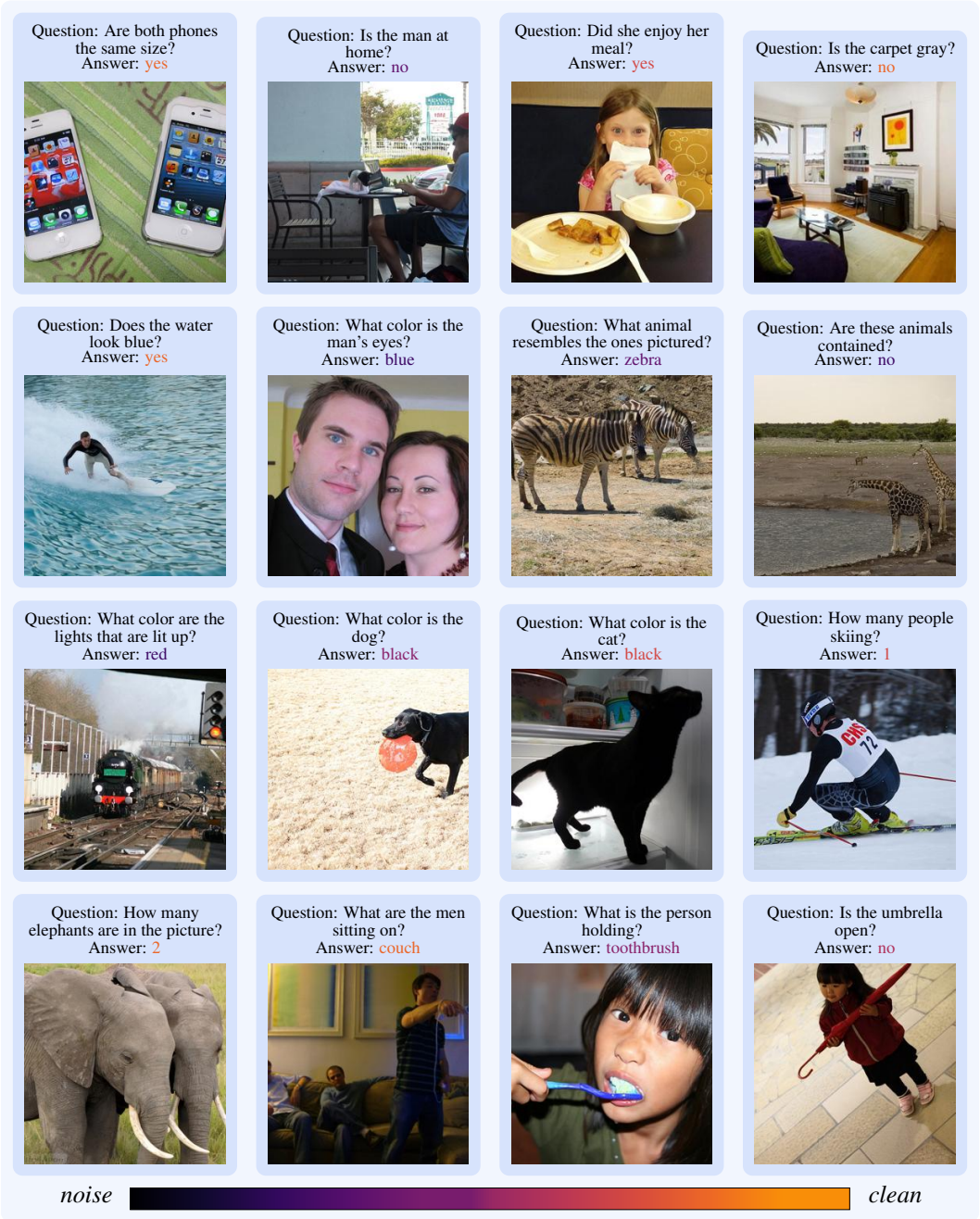


Figure 20: VQAv2 predictions from the SD3 FullFlow VQA finetune on unseen validation images. Each tile shows the input image, the question, and the model's predicted answer.

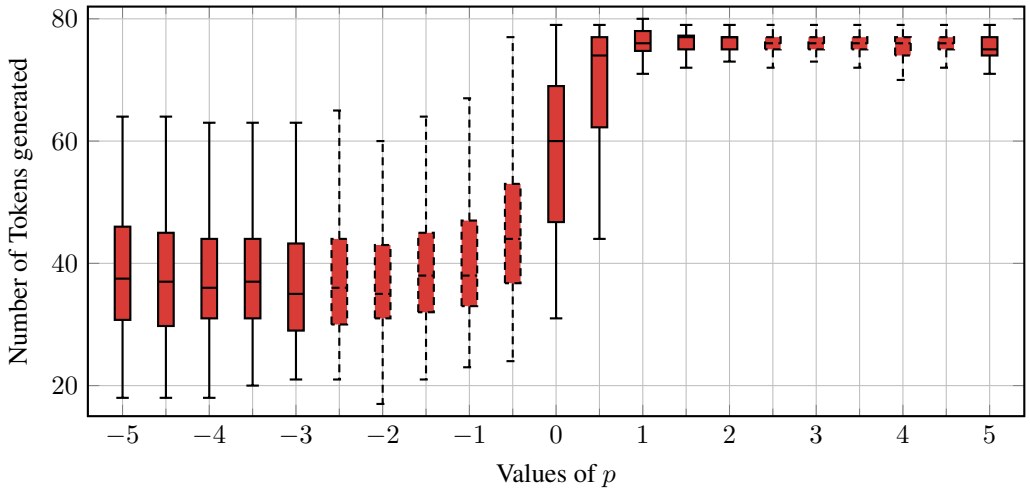


Figure 21: Caption-length distribution under joint image–text sampling as a function of the trajectory parameter p in $\tau = t^{2^p}$. Boxes show the interquartile range (Q1–Q3) with the median; whiskers extend to the min/max sample. Larger p (image-first) yields longer captions, consistent with the qualitative examples below.

Figure 22: Joint image–text samples from FullFlow (seed 1/5) for $p \in \{-5, -2.5, 0, 2.5, 5\}$, shown left to right. Negative p denoises text first, $p = 0$ denoises both modalities together, and positive p denoises the image first.

Figure 23: Joint image–text samples from FullFlow (seed 2/5) for the same trajectory sweep as Figure 22.

Figure 24: Joint image–text samples from FullFlow (seed 3/5) for the same trajectory sweep as Figure 22.

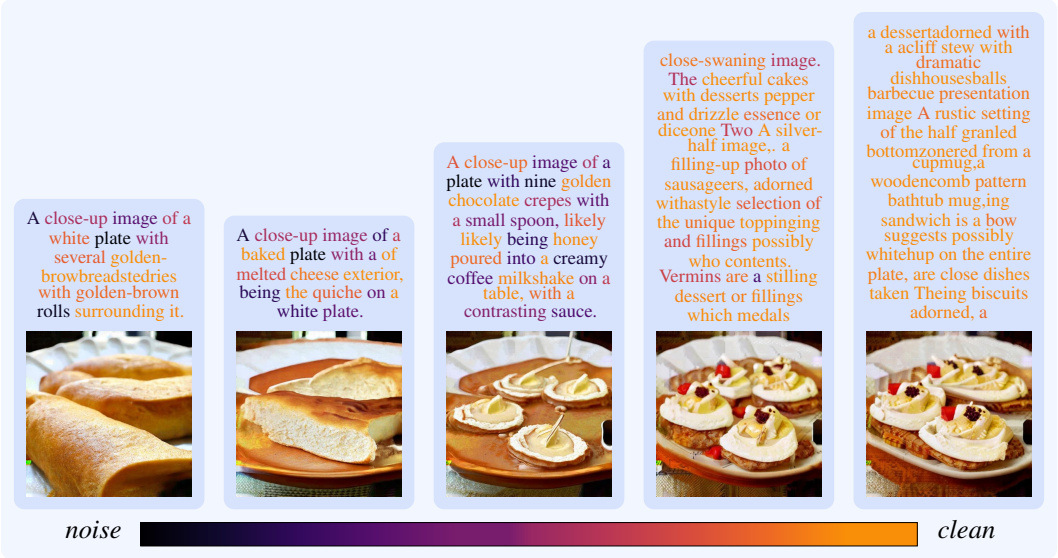


Figure 25: Joint image–text samples from FullFlow (seed 4/5) for the same trajectory sweep as Figure 22.

Figure 26: Joint image–text samples from FullFlow (seed 5/5) for the same trajectory sweep as Figure 22.

NeurIPS Paper Checklist

1. **Claims**
   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?
   Answer: [Yes]
   Justification: Contributions **C1–C3** of Section 1 are substantiated by the matched-baseline experiments in Section 5.3 and Table 4 (full numerics in Table 9), the FLUX.1-dev transfer in Section 5, and the downstream-VQA results in Section 5.4 and Table 3. The scope of these claims (compute-feasible proof of concept, not peak end-to-end performance) is explicitly stated in Section 6.
   Guidelines:
   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.
2. **Limitations**
   Question: Does the paper discuss the limitations of the work performed by the authors?
   Answer: [Yes]
   Justification: Section 6 acknowledges that this is a compute-feasible proof of concept at 256×256 on a $\sim$280k LAION subset (Sections B and B.3), addressed by data-, parameter-, and compute-matched baselines (Section B.8), with image$\rightarrow$text quality bounded by the single-reference Moondream captions and OKVQA bounded by pretraining-data scale (Section 5.4 and Table 3). Natural extensions to higher resolution, larger datasets, and instruction tuning are explicitly deferred to future work in Section 6.
   Guidelines:
   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.
3. **Theory assumptions and proofs**
   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Theorems E.1 and E.2 are stated and proved in full in Section E, with all assumptions (Gaussianity and prescribed covariance for MSE; one-hot targets and prescribed expected agreement σ for cross-entropy) made explicit in the theorem statements. The bounds are visualized in Figure 9 and consistent with the empirical $\lambda_{\text{txt}} \approx 0.04$ obtained by the adaptive balancing rule in Sections 3.4 and 5.2.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Architecture, objective, and stabilizers are fully specified in Sections 3, 3.2, 3.4, F and G, with complete hyperparameters for both FullFlow and the matched DualDiff.-LoRA baseline in Table 5 and Sections B and B.8 and evaluation protocols in Sections B.9, B.10 and C. Code, the 1.9k-prompt evaluation suite (Section B.2), and the SD3/FLUX.1-dev LoRA checkpoints will be released publicly upon publication.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?
Answer: [No]
Justification: To preserve double-blind anonymity and because the trained checkpoints exceed supplemental-material size limits, code, checkpoints, and the 1.9k-prompt suite (Section B.2) are withheld at submission and will be released publicly upon publication. Reproduction from the public SD3-M [12] and FLUX.1-dev [25] backbones is fully supported by the documented data pipeline, hyperparameters, and protocols in Sections B, B.8 and C and Table 5.
Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**
Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?
Answer: [Yes]
Justification: Section 5.1 summarises the setup, and Table 5 and Sections B and B.3 list every training detail (optimizer, learning rates, schedule, gradient clipping, batch size, LoRA configuration, dataset pipeline, and resolution rationale) for both FullFlow and the matched DualDiff.-LoRA baseline (Section B.8). Test splits, the prompt suite, and the per-metric evaluation protocols are documented in Sections B.2, B.9, B.10 and C.
Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**
Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?
Answer: [Yes]
Justification: The corruption-schedule ablation reports bootstrap 95% confidence intervals over the 5k held-out validation split for CIDEr in Figure 5 and for four additional metrics in Figure 14, with raw mean $\pm$ std across the five switch-point seeds tabulated in Table 8 and full per-metric main-comparison numbers in Table 9. Training uses a fixed seed of 42 (Table 5), so the intervals capture variability from evaluation-split bootstrap resampling, as detailed in Section C.
Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Section D and Table 6 report hardware (RTX A5000, 24 GB, BF16), GPUs per run, wall-clock per run, and totals per run group, summing to 1,212 reported GPU-hours. Section D also explicitly states that preliminary and failed configurations not used in the paper are excluded from this total.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work uses openly released SD3-M and FLUX.1-dev backbones with a public LAION subset (Sections B and C), involves no human subjects or PII, and preserves anonymity at submission. Dual-use risks and recommended mitigations are addressed in the broader-impact paragraph of Section 6.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: The broader-impact paragraph of Section 6 discusses both positive impacts (lowering the compute barrier for bidirectional image–text generation, enabling interactive applications) and negative ones (synthetic image–caption misuse, biased captions, misleading edits, dataset contamination). It also recommends mitigations in line with NeurIPS guidance: provenance tracking, watermarking, and content filtering aligned with the underlying base models.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.

- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The released artifacts are LoRA adapters and lightweight text heads (Sections B and 3.2) on publicly released SD3-M [12] and FLUX.1-dev [25] backbones; they introduce no new generative capacity beyond what those open-weight models already provide and inherit their existing license-based safeguards. We do not redistribute the underlying LAION images (Section B), so dedicated additional safeguards are not applicable.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: All backbones, datasets, and methodological references (SD3-M, FLUX.1-dev, LoRA, LAION, Edit Flows, rectified flow, OneFlow, Dual Diffusion) are cited where introduced in Sections B, B.8, 3, 3.2 and 5.1 and used under their public licenses. The evaluation toolkit (clean-fid, CMMD, CIDEr, BERTScore, CLIP, VQAv2, VizWiz, OKVQA) is cited with library names, model identifiers, and URLs in Sections B.2 and C.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: We plan to release SD3-M and FLUX.1-dev FullFlow LoRA checkpoints, the 1.9k-prompt text-to-image evaluation suite, and the training/evaluation code (including the DualDiff.-LoRA reproduction), all of which are documented in the paper (Sections B, B.2, B.8, C, 3.2 and F and Table 5). Usage instructions and license information will accompany the public release through the project page upon publication.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: This work does not involve crowdsourcing or research with human subjects; all data come from publicly released datasets and a model-generated captioning pipeline (Sections B, B.2 and C).

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve human subjects (Sections B and C), so no IRB or equivalent review was required.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.

- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs were used only as a writing and code-review aid and are not part of the core methodology developed in Sections 3, 3.4, E and 5. The Moondream captioner used to label the LAION subset (Section B) is a fixed public data-labelling tool, not a methodological LLM component.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.